Algo 笔记本

yuanhao liu

Exported on 09/24/2025

Table of Contents

1 课题内容 4
2 课题要求 5
3 NEON 优化方案 6
4 实现思路笔记 7
5 刷机刷固件笔记 26
6 笔记 32
7 项目笔记 53
7.1 25年08月27日 合作大作业 53
7.1.1 总结 53
7.1.2 题目要求 53
7.1.3 实现思路 53
7.1.4 思路【中期】 54
7.1.5 调研笔记 55
7.2 25年08月18日 AMS作业：编译一个AMS模块 56
7.3 25年08月13日 竞品对比 56
8 练手任务 2025.7.28-2025.8.4 58
9 竞品测试笔记 59
9.1 25年08月 新人工作日报留档 59

课题背景

音频算法NEON优化：当前音频算法在算法原理上进行过多轮优化，原理上已基本达到最优。后续可进行NEON优化，进一步降低CPU消耗。 降低CPU消耗有助于应对芯片降本，CPU余量可用于新增其他音频功能。

1 课题内容

1. 调研neon指令使用方法
2. 实现neon指令本地仿真运行
3. 完成neon代码开发
4. 实现实机运行

2 课题要求

1. 输出neon调研报告
2. 输出neon仿真工具及使用说明
3. 实际运行，双向语音通话CPU降低20%
4. （补充） 设计一个rtf模块，可以直接使用，统计各模块的rtf。 工作意义：仿真profile不能反应实际芯片；实测只有线程cpu，看不到各模块占用。
    a. 实时因子（RTF），real time factor，实时因子，一般在实时语音识别过程中的评价延迟标准。RTF 定义为模型解码时间与输入音频持续时间的比率。
    b. 相关计算公式

二、CPU占用率

CPU占用率，算法处理所需时长除以CPU总时长。

cpu最小计算时间为时钟周期*，cycle。1秒内cpu拥有的时钟周期总数对应着cpu的主频。例如，2GHz主频的cpu共有2G个时钟周期，即计算周期，每个时钟周期为1/2G 秒，故，CPU占用率的计算为：在一段时间内，程序（算法）使用的cycle总数除以这段时间内cpu的cycle总数。公式如下：

$$CPUusage = \frac{CyclesUsage}{TotalCPUcycles}$$

三、两者的关系

RTF的分子和CPU占用率的分子有着相同的物理意义，指算法*处理所需的时间：

1、RTF的分子是算法处理所需的时间；

2、CPU占用率的分子是算法处理所需的时钟周期数，由于每个时钟周期时长等于1/f，其中f为cpu主频，所以时钟周期数*乘以时钟周期时间等于算法处理所需的时间。

两者分子间的关系如下：

$$ProcessTime = CyclesUsage * 1/f$$

    c.
    d. CPU time 指的是计算机处理器在执行一个特定程序时花费的时间，也就是程序在处理器上实际运行的时间。
    Clock time（也称为 wall time 或 real time）指的是程序从开始执行到结束所花费的时间，包括了等待资源、I/O 操作等等与 CPU 时间无关的时间。
    Real time（也称为 wall time 或 clock time）指的是从开始执行程序到程序完成所花费的时间，包括了 CPU 时间和与 CPU 时间无关的时间。
    Wall time（也称为 clock time 或 real time）和 Real time 意义相同，都是指程序从开始执行到结束所花费的时间，包括了 CPU 时间和与 CPU 时间无关的时间。

3 NEON 优化方案

1. 关于实现neon指令本地仿真运行： 使用 NEON2SSE.h 仿真。 Ne10 需要的环境暂不确定。
2. 关于“调研neon指令使用方法”：ARMV7+NEON汇编指令详解中文版.pdf[1]
3. 关于“输出neon调研报告”：“还有一点：开发的时候关注音频写neon的特别之处，记录下来，尽可能多。后续答辩会有软件同事，他们可能比你更懂neon，你要体现你工作的独到之处，最好是难点，或者一些差异也行。比如：音频处理中的FFT、自适应滤波 这些操作，计算量占用大，本身又可大量并行，做neon化的收益高。”
4. 关于“实际运行，双向语音通话CPU降低20%”：

1 https://pdconfluence.tp-link.com/download/attachments/157614785/ARMV7%2BNEON%E6%B1%87%E7%BC%96%E6%8C%87%E4%BB%A4%E8%AF%A6%E8%A7%A3%E4%B8%AD%E6%96%87%E7%89%88.pdf?api=v2&modificationDate=1756867171430&version=2

4 实现思路笔记

1. 论文从这里开始完全看不懂。为什么要设 delta， 为什么要算 delta(n+1)/delta(n)，式子(6) 怎么推出来的？？
2. 看起来 confluence 的 图片 功能不太靠谱。。。
3. micro SD 之间差异很大；不要往机身存储dump，会卡死；卡死一段时间后，有概率自动 reset 救活。
4. 编译固件报错？答：重新拉一遍节点。make clean。再编译。（可能 src 和 build_dir 的代码不完全一致）
5. 怪兽音效杂音？答：因为app记录的采样率和DUT不同，导致app听到的结果是错的。
    a. 解法：app清除缓存，reset设备，重新连一遍。
6. reset 的三种方法
    a. 手机app上移除设备
    b. 摄像头reset键
    c. 串口板 dbg_cmd reset
7. 升级报错？UpgradeFirmware接口HTTP装填码不符合预期：0？
8. 答：看起来像是对应进程休眠了。拔电重启摄像头后正常。
9. reset密码TPL075526460603
10. 快速访问docker对应文件夹 cd NVMPs/c225v2-neon/NVMP/nvmp z
11. 磁盘被吃满了。。。导致 vscode 连不上虚拟机。
12. 使用 Tapo_C225v2_en_1.1.2_Build_250916_Rel.7048n-dbg_up_boot 刷机，其中 dbg 表示开启串口板
13. 建议每天重启虚拟机，不然磁盘碎片可以多到把 vscode 卡死
14. 报错 /home/tplink
    Cloning into 'NVMP'...
    The authenticity of host '[gerrit-ce.tp-link.com[2]]:29418 ([10.176.69.107]:29418)' can't be established.
    ECDSA key fingerprint is SHA256:94zokm8g9MOxm56wKUpvlohudHHs+WfWn+YsZ1pWeDM.
    Are you sure you want to continue connecting (yes/no/[fingerprint])? 怎么解？
15. 答：通过在bash中配置home，避免调用到 root 的home= / root / .ssh ，就可以解
16. 尝试使用 build_main 直接编译固件。 失败，发现脚本里写死“nvmp_source_dir” 的目录名称，而我的文件结构是 NVMPS/c220v2/NVMP/nvmp
17. ```
    ./scripts/build_main.sh -p c225v2 -s all
    ```
18. docker相关学习笔记

2 http://gerrit-ce.tp-link.com
    ```

19.

```
查询容器docker名字
# docker ps -a --format '{{.Names}}'

进入容器目录并进入可交互bash
# docker exec -it "nvmp_builder_16.04" bash -c "
    cd /nvmp_source_dir/nvmp || exit 1
    exec bash

尝试跳过默认编译选项，失败。怀疑是 makefile 具体执行的命令，并不能从管道取到 "\n" 的输入，
需要对特定的命令做 "-y"
# yes "" | make PR_NAME=$PR_NAME V=s -j1


docker inspect nvmp_builder:16.04 --format '{{json .Mounts}}' | jq
```

20. 如何把 NVMPs 全部挂载到 docker 而不是只挂载一个 nvmp_source_dir ？

21.

```
# my_build_docker.sh 在 【TODO】
# 在任意 NVMP 目录下 sudo bash (绝对路径)my_build_docker.sh
#!/bin/bash

SOURCE_DIR=`git rev-parse --show-toplevel`
TIMESTAMP=`date +%Y%m%d%H%M%S`
FROCE_BUILD=0
CLEAN_BUILD=0
SIGN_FIRMWARE=0
REPO_DIR="/home/tplink/audio_demos/NVMPs"
[[ "$UBUNTU_VER" == "" ]] && UBUNTU_VER=16.04

while [[ $# -ge 1 ]]; do
    case $1 in
        -B)
            FROCE_BUILD=1
            shift
            ;;
        -C|--clean)
            CLEAN_BUILD=1
            shift
            ;;
        -S|--sign)
            SIGN_FIRMWARE=1
            shift
            ;;
        *)
            [ "$PRODUCTS" == "" ] && PRODUCTS="$1" && shift && continue
            [ "$TARGET" == "" ] && TARGET="$1" && shift && continue
            shift
            ;;
    esac
done
```

```
# use default products for specified platform
echo "Products: ${PRODUCTS}"
echo ${PRODUCTS} > $SOURCE_DIR/nvmp/docker/.products
echo ${TARGET} > $SOURCE_DIR/nvmp/docker/.target

# builder name
[[ "$BUILDER_NAME" == "" ]] && BUILDER_NAME=nvmp_builder_${UBUNTU_VER}
echo "Builder: ${BUILDER_NAME}"

# stop and clear previous docker container
if [[ $FROCE_BUILD -eq 1 ]]; then
    docker stop $BUILDER_NAME 2>/dev/null
    docker rm $BUILDER_NAME 2>/dev/null
fi

# clean workspace before building
if [[ $CLEAN_BUILD -eq 1 ]]; then
    repo forall -c "git clean -dfx -e .repo"
fi
# get the status of container
CONTAINER_INFO=`docker ps -a | grep $BUILDER_NAME | grep $UBUNTU_VER`
if [ "$CONTAINER_INFO" == "" ]; then
    # docker server
    DOCKER_SERVER="rdsource.tp-link.com:8088"

    # try to load the nvmp builder image
    BUILDER_IMAGE=`docker image list | grep nvmp_builder | grep $UBUNTU_VER`
    if [ "$BUILDER_IMAGE" == "" ]; then
        # 下载镜像源文件
        SOURCE_LIST_URL=http://rdsource.tp-link.com/files/Ubuntu/${UBUNTU_VER}/
sources.list
        SOURCE_LIST_FILE=${SOURCE_DIR}/nvmp/docker/sources.list
        wget ${SOURCE_LIST_URL} -O ${SOURCE_LIST_FILE} || rm $
{SOURCE_LIST_FILE}

        # login docker server
        docker login -u docker -p docker $DOCKER_SERVER
        docker pull $DOCKER_SERVER/ubuntu:${UBUNTU_VER} || exit -1

        # create new container
        echo "now creating the nvmp builder container ..."
        docker run -i -d --privileged --dns 172.29.1.1 --name nvmp_builder_$
{UBUNTU_VER} \
            -v ${REPO_DIR}:/NVMPs -v ~/.ssh:/ssh \
            $DOCKER_SERVER/ubuntu:${UBUNTU_VER} bash

        # prepare env
        docker exec -i nvmp_builder_${UBUNTU_VER} /NVMPs/c225v2/NVMP/nvmp/
docker/prepare_env.sh
        # /home/tplink/audio_demos/NVMPs/c225v2/NVMP/nvmp/docker/prepare_env.sh
根据相对目录修正
```

```
        # check the status
        BUILD_RET=${PIPESTATUS[0]}
        if [[ ${BUILD_RET} -ne 0 ]]; then
            echo "builder create failed: ${BUILD_RET}"
            docker container stop nvmp_builder_${UBUNTU_VER} > /dev/null
            docker container rm nvmp_builder_${UBUNTU_VER} > /dev/null
            exit ${BUILD_RET}
        fi

        # save as image
        docker commit nvmp_builder_${UBUNTU_VER} $DOCKER_SERVER/nvmp_builder:$
{UBUNTU_VER}
        docker container stop nvmp_builder_${UBUNTU_VER} > /dev/null
        docker container rm nvmp_builder_${UBUNTU_VER} > /dev/null
    fi

    # get the ccache dir
    CCACHE_DIR=`ccache -s 2>/dev/null | grep directory | awk '{print $3}'`
    if [[ "$CCACHE_DIR" == "" ]]; then
        echo "no ccache installed, create the default ccache dir"
        CCACHE_DIR="$HOME/.ccache" && mkdir -p $CCACHE_DIR
    fi
    echo "ccache dir is: $CCACHE_DIR"

    # load container
    docker run -i -d --privileged --dns 172.29.1.1 --name $BUILDER_NAME \
        -v ${REPO_DIR}:/NVMPs \
        -v ${CCACHE_DIR}:/home/tplink/.ccache \
        $DOCKER_SERVER/nvmp_builder:${UBUNTU_VER} bash
fi
```

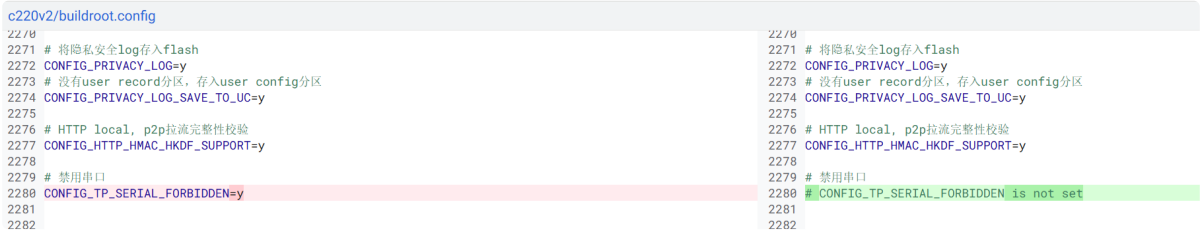
22. [fac_test]bc_develop_220927:ae996b9 这个fac_test 是不是只和测试有关？ 脚本里没这个的相对位置 这个本地编不用拉
23. 本地自测 commitid 【前4个】
24.

```
tplink@tplink-vm:~/audio_demos/NVMPs/c225v2/NVMP/nvmp/ams_package/ams/src/aqi/
aecns/aec$ git log --oneline
b2f6f3cac (HEAD -> nvmp_release_1.8) [all] 修复固件编译 warning
e23c68252 [all] 引入 NE10 FFT 库并优化 AEC3 模块
2eb5424e9 [all]删除 st->motor_gain_linear = 1;
bad2601b6 [all] AQI: aec3 add simplified MDF echo cancellation algorithm
git cherry-pick bad2601b6
git cherry-pick 2eb5424e9
git cherry-pick e23c68252
git cherry-pick b2f6f3cac

#0918补充 【仓库=c225v2-neon 】
```

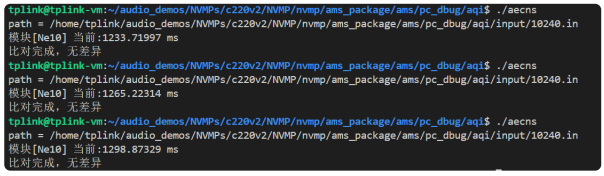
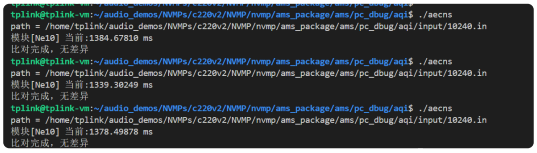
```
b22b67190 (HEAD -> nvmp_release_1.8)  [all] WIP add TODOs & delete float_diff()
& setNEONFlag()
6950051a9 [all] 修复固件编译 warning
08a0a38e8 [all] 引入 NE10 FFT 库并优化 AEC3 模块
3df72da96 [all]删除 st->motor_gain_linear = 1;
c98253a03 [all] AQI: aec3 add simplified MDF echo cancellation algorithm
a6e369269 audio dump

git cherry-pick  a6e369269
git cherry-pick  c98253a03
git cherry-pick  3df72da96
git cherry-pick  08a0a38e8
git cherry-pick  6950051a9
git cherry-pick  b22b67190
```

25. 提测节点拉不到怎么办？fatal: ambiguous argument 'c11d884': unknown revision or path not in the working tree.
    Use '--' to separate paths from revisions, like this:
    'git <command> [<revision>...] -- [<file>...]'
    product_config c11d884 node wrong!!!
26. 复制文件夹之前，还是稍微删除下 build_dir 。不然很慢。
27. 如何查看样机机型？ 连上串口，命令行左侧会显示机型。也可以和 tapo app 对照。
28. 机子是好的，串口版没输出怎么办？答：可能是发布版固件把串口板禁用了，找软件要一个类似 http://gerrit-ce.tp-link.com/c/IPC/NVMP/PRODUCT/+/327403 的commit ，在 product_config 的目录下，cherrypick这个 commit。 如果你的cherrypick新建了文件，说明你选的目录是错的。 e.g. /home/tplink/audio_demos/NVMPs/c220v2/NVMP/nvmp/product_config
29. 

30. 如果 make -j1 翻不到 error 的具体位置？ 答： 再次make，可能只是代码具体位置的 error 被刷上去了
31. 主要 commit
    a. http://gerrit-ce.tp-link.com/c/algorithm/video/nvmp_docker/+/330090 函数实现 calc_array
    b. http://gerrit-ce.tp-link.com/c/algorithm/video/nvmp_docker/+/312395 王昶依赖
    c. http://gerrit-ce.tp-link.com/c/algorithm/video/nvmp_docker/+/330689 引入 Ne10-FFT，编写函数c_neon_fft/ifft/init/destroy
    d. http://gerrit-ce.tp-link.com/c/algorithm/video/nvmp_docker/+/330091 NEON2SSE.h 仿真环境和 makefile 编译选项 【包含 calc_array 的bugfix】
32. AU 比较音频频谱方法：直接同步时间轴后，看频谱 → 反相其中一个音频 → 混合式粘贴（重叠） 到另一个音频
33. 混合后结果如图，在右侧坐标轴滚轮放大，可以观察到峰值差异是 6 ，相当于 6/32768 = 2e-4 的误差。符合预期。

34. 多读 gcc 编译warning 就能识别很多编译陷阱
35. 注意 init 和 destroy 会占一半时间！【以下使用高精度时间测试】
36.

ne10 -> neon_fft【仿真】	ne10 -> c_fft
-O3	-O3
	
SPX-O3	
	

37. 为什么 free 的时候直接 free(cfg) 即可？答：因为 cfg 申请的空间是整块 malloc 的，源码如下。
38.

```
    ne10_fft_r2c_cfg_float32_t st = NULL;
    ne10_int32_t ncfft = nfft >> 1;

    ne10_uint32_t memneeded = sizeof (ne10_fft_r2c_state_float32_t)
                              + sizeof (ne10_int32_t) * (NE10_MAXFACTORS * 2)
/* factors */
                              + sizeof (ne10_fft_cpx_float32_t) * ncfft
/* twiddle */
                              + sizeof (ne10_fft_cpx_float32_t) * (ncfft / 2) /
* super twiddles */
                              + sizeof (ne10_fft_cpx_float32_t) * nfft
/* buffer */
                              + NE10_FFT_BYTE_ALIGNMENT;                    /*
64-bit alignment */

    st = (ne10_fft_r2c_cfg_float32_t) NE10_MALLOC (memneeded);
```

39. 测试 FFT 后 IFFT 与初始 input 的偏移量。input=27.in

40.
```
最大绝对值偏移量
spx: -0.008789 【初始input=0.000000】
neon_fft: -0.012695 【初始input=0.000000】
```

41. C 语言的 struct 强转 float* 是很常用的方法。对应 warning 如下，无需处理。你看到的这种 struct {float r; float i; } <-> float* 强转，在很多 FFT 库 里确实用得很广，尤其是那些追求 SIMD 优化的实现。像 FFTW、KissFFT、ARM NE10、Intel MKL 这些 FFT 库，都在内部或接口层面用过这种技巧。
它利用了「复数就是一对 float」这一事实，把 struct/array/float* 在内存上对齐起来，从而让 SIMD 或批量处理更方便。

42.
```
fftwrap.c:140:48: warning: passing argument 2 of ‘ne10_fft_c2r_1d_float32_neon’
from incompatible pointer type [-Wincompatible-pointer-types]
  140 |         ne10_fft_c2r_1d_float32_neon(realData, tempComplex, cfg);
      |                                      ^~~~~~~~~~~
      |                                      |
      |                                      float *
```

43. 在代码原型里 FFT 明明结果对齐了，放进真实代码里结果却差很远？答：检查 init 和 fft_n 和 scale，这几个值在 FFT 有好几种定义方法。例如 ncfft*2 = nfft

44. 如何 Debug GCC 的指针？尤其是 指针嵌套指针，各种 * 和 & 很容易写错。 最好的办法是每一步念出来“我这里取得是值还是地址，取的是具体哪一层”。 Debug 的时候，把对应指针的地址完整记在旁边，看清楚地址在 传参 的过程中，变化是否符合预期。一般出现“fff” 开头的指针，就是取到指针的地址了，例如 &ptr ，ptr 本身是个指针。

45.
```
Locals
  t = 0x5555556e2ba0
  scale = 0.0009765625
  n = 1024
  cfg = 0x5555556e2ba0
> realData = 0x5555556d36e0
> compData = 0x5555556e0b70
  nfft = 1024
```

46. **fft 结果做 ifft 无法直接还原原始信号，这次的误差到了 1e-3** ，尤其是稀疏信号： 一大堆 0 ，有部分值到达 ±32468 以上的信号，最大误差达到 1e-2。考虑以下方案：
    a. 直接做 32768 的截断。会导致算出的频域和原来的差异更大。反而需要在 ams 源代码中做更多“为了提高 FFT 精度而做的对应改动”
    b. 对稀疏信号特殊处理，以降低 max_diff。
    c. 认为是可以接受的。通过最后得到的音频判断 该 FFT 的影响。经查，在 input=(int)0~32768 的动态范围内，源代码也会带来 1/32768 的精度误差。

d. 转 双精度浮点数。显然本末倒置，速度巨慢。
e. 转 NTT。属于想象力太丰富了，不可做。
f. 原理上讲， FFT 和 IFFT 的代码是等价的。既然 FFT 的误差做到了优于 spx_fft ，那么 IFFT 的误差应当是自然优于 spx_fft 的。

47. 作者还真的在 test_fft 里面下毒了。存个疑，没测 ifft 还。 直接做音频仿真了

48. test_fft.c

C: > Users > admin > AppData > Local > Temp > 162e3093-4571-4ed3-8ea5-37038d721fed_n

```
104    #endif
105
106    #if __ARM_NEON
107        ne10_fft_c2r_1d_float32_neon(inData, outData, cfg);
108    #else
109        ne10_fft_r2c_1d_float32_c(outData, inData, cfg);
110    #endif
111
112    #if DEBUG
113        printf("After IFFT\n");
```

49. 很诡异， 为什么编译并运行， 会比直接运行快 10ms ；；已经确认了不是 “CLOCK_MONOTONIC” 高精度时钟带来的，也不是 freopen 带来的 ；； 如果是内存相关的因素，时延不会差这么大吧 ？
    a. 答： Linux 调度粒度：Linux 调度时间片一般是 1–10ms。所以你看到的差距正好在这个量级，很可能是进程在启动时被 调度挂起/唤醒，导致执行时间有抖动。
    b. 解决方案：同时测试 10000 条数据，把总运行时间拉长到 1s 以上。这样抖动带来的影响就可以忽略了。

50.
```
tplink@tplink-vm:~/audio_demos/NVMPs/c220v2/NVMP/nvmp/ams_package/ams/pc_dbug/aqi$ python compile.py aec3 ns3 agc tpaec3ns
3 && ./aecns
模块[spx_fft_main] 当前:7.36415 ms
比对完成，无差异
tplink@tplink-vm:~/audio_demos/NVMPs/c220v2/NVMP/nvmp/ams_package/ams/pc_dbug/aqi$ ./aecns
模块[spx_fft_main] 当前:17.97410 ms
比对完成，无差异
tplink@tplink-vm:~/audio_demos/NVMPs/c220v2/NVMP/nvmp/ams_package/ams/pc_dbug/aqi$ ./aecns
模块[spx_fft_main] 当前:16.96098 ms
比对完成，无差异
tplink@tplink-vm:~/audio_demos/NVMPs/c220v2/NVMP/nvmp/ams_package/ams/pc_dbug/aqi$ ./aecns
模块[spx_fft_main] 当前:17.09441 ms
比对完成，无差异
tplink@tplink-vm:~/audio_demos/NVMPs/c220v2/NVMP/nvmp/ams_package/ams/pc_dbug/aqi$ ./aecns
模块[spx_fft_main] 当前:18.30852 ms
比对完成，无差异
tplink@tplink-vm:~/audio_demos/NVMPs/c220v2/NVMP/nvmp/ams_package/ams/pc_dbug/aqi$
```

51. 注意两种写法的差异

```
            float eps = 9e-5;
            float val = (max(1.0f, abs(outData[i]) + abs(fft_out[i])))*eps; //
支持 num*1e-6 级的误差
            double valdiff = abs((double)outData[i]-(double)fft_out[i]) /
(abs((double)outData[i]) + abs((double)fft_out[i]));
```

```
            if(abs((double)outData[i]-(double)fft_out[i]) > val) // 预期支持
 valdiff<=1e-6 级的误差
            if(valdiff  > eps) // 预期支持 valdiff<=1e-6 级的误差
```

使用 valdiff 直接计算比例的话，当 (abs((double)outData[i]) + abs((double)fft_out[i]) << 1 时，(double)outData[i]-(double)fft_out[i]) 由 float 带来的 1e-7 误差也会被放大，实践中甚至放大到 1e-1 的级别！！而使用 val 这种写法 对变量精度的表示压力小很多

52. 多用**编译并运行**“cmake --build . && sudo ../bin/test_fft”，有时保存文件后敲得太快也会导致结果不对。
53. 仿真测试，测速均使用 clock() ，测试input=10584.in[3] ， 包括约 10584 次 1024 点FFT。时间差距对比：

ne10 -> neon_fft	ne10 -> c_fft	ams_spx_fft
2365ms	2504ms	3155ms
+33.4%	+26.0%	100%

54. 阶段性成果：neon_fft 兼容 NEON2SSE.h 的仿真环境，但现在在小数点后4位有明显误差。没有测完所有测试组。预期要求是 abs(x-y) / (abs(x)+abs(y)) <= 1e-5 （float本身会带来 1e-6 左右的误差）。现在支持 9e-5 的误差。 基本排除 cfg 没有重置带来的误差。和 python 的高精度 FFT 结果（视为“标准结果”）对比后发现，neon-fft 和 原本的 kiss-fft 得到的 float 结果，几乎是随机分布在 标准结果的两侧，且与标准结果的差异在 1e-6 范围内，但是经过四舍五入之后，可能带来小于 3e-6 的误差。至此可以认为是 float 本身带来的难以优化的精度误差。
55.

```
python_highres_fft = -6.559676346107324e-06
spx_fft = -0.000006
neon_fft = -0.00000739

python_highres_fft = -0.0019030693116829922
spx_fft = -0.001903
neon_fft = -0.00190169
```

3 http://10584.in

56.

```
[100%] Linking C executable ../bin/test_fft
[100%] Built target test_fft
tplink@tplink-vm:~/audio_demos/neon-fft-main/build$ sudo ../bin/test_fft
共 10584 组，比对完成，无差异
耗时：2400.830000 ms
tplink@tplink-vm:~/audio_demos/neon-fft-main/build$ sudo ../bin/test_fft
共 10584 组，比对完成，无差异
耗时：2321.507000 ms
tplink@tplink-vm:~/audio_demos/neon-fft-main/build$ sudo ../bin/test_fft
共 10584 组，比对完成，无差异
耗时：2365.466000 ms
tplink@tplink-vm:~/audio_demos/neon-fft-main/build$
```

```
Diff886:1.082751 1.082774
Diff904:-6.309006 -6.308868
Diff915:-0.530418 -0.530380
Diff930:0.088284 0.088295
Diff932:1.353443 1.353512
Diff938:0.868078 0.868040
Diff942:-1.760521 -1.760452
Diff947:0.278111 0.278095
Diff950:-0.285488 -0.285438
Diff961:-1.438459 -1.438498
Diff962:-1.496735 -1.496700
Diff965:-0.668220 -0.668204
Diff966:-0.173982 -0.173948
Diff968:-1.033928 -1.033899
Diff976:-0.977634 -0.977594
Diff980:0.804995 0.805017
Diff981:0.179308 0.179331
Diff986:-0.731311 -0.731330
Diff994:-1.103040 -1.103110
Diff997:2.143633 2.143682
Diff1000:-1.866376 -1.866337
Diff1002:-0.231568 -0.231482
Diff1005:0.475780 0.475733
Diff1018:-0.168895 -0.168917
Diff1021:2.906255 2.906181
Diff1022:-0.546020 -0.546034
```

```
94          // printf("\n");
95          for(int i = 0; i < fft_n; i++)
96          {
97              float val = (max(1.0f, abs(outData[i]) + abs(fft_out[i])))*9e-5; // 支持 num*1e-6 级的误差
98              if(abs(outData[i]-fft_out[i]) > val)
99              {
100                 printf("%d : Diff%d:%f %f\n" tot  i  outData[i]  fft_out[i]);

问题 7   输出   调试控制台   终端   端口   GITLENS

      |      ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
[100%] Linking C executable ../bin/test_fft
[100%] Built target test_fft
共 10584 组，比对完成，无差异
耗时：2510.521000 ms
tplink@tplink-vm:~/audio_demos/neon-fft-main/build$
```

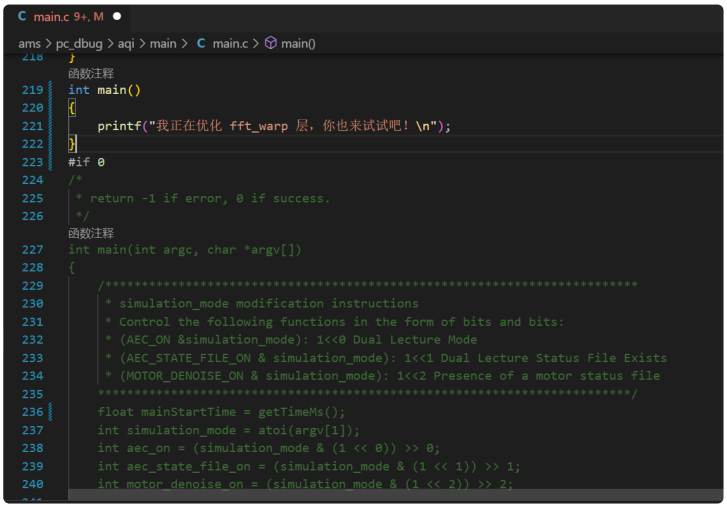
57.

来自“语音算法组”公众号 neon-fft-main

编译并运行

```
# cd build/ # 建build目录
# cmake ..
# cmake --build .
[100%] Linking C executable ../bin/test_fft
[100%] Built target test_fft
```

```
--build 是 CMake 的构建模式选项，指示 CMake 调用底层构建工具（如 make 或 ninja）执行编译操作。 --build 是 CMake 的跨平台命令，自动适配本地构建工具（如 Linux 下调用 make，Windows 下调用 msbuild）
```

58. [Big Input] tot=78102 。输入信号 max: 165923.781250, min: -143769.875000 ， 按照 float 的 7 位有效数字算，计算 fft 是一定会有精度损失的 。 输出信号 max: 8339.438477, min: -8612.413086。 if(tot_fft == 100489) output
59. 不要把咖啡和其他饮料放在右手侧，尤其是鼠标碰得到的地方。
60. ns2 也有个类似的 fftwrap.c 【ams/src/aqi/aecns/ns/ns2/fftwrap.c】应该是弃用的，不用看。 和 aec3 的 fftwrap.c 的实现略有不同。
61. 如何快速测试你的新模块？ 在 main 添加对应的调用， 推荐使用 #if 0 完成整段注释来规避 /**/ 不能嵌套整块注释的问题。
62. 

63. 如何控制 PC_DBUG 和 NEON_NO_SUPPORT 等宏定义选项？ 答：在编译选项添加 “ -DPC_DEBUG ”，其中“ -D” 表示宏定义， -DMACRO_NAME=value # 定义带值的宏（等价于 #define MACRO_NAME value）
64. ```
cmd = "gcc -o aecns -DPC_DEBUG " + GCC_I_DIRS + " "
```
65. FFT 的交错实数存储
```

66.

1. 实数FFT输出的特殊性

对于 **N点实数FFT**，输出频谱具有共轭对称性（Hermitian symmetry），因此有效频点仅需存储 N/2 + 1 个复数（含直流和Nyquist分量）。常见的存储方式有两种：

(1) 标准复数数组

- 长度：N/2 + 1 的复数（如 fftwf_complex）
- Nyquist位置：显式存储在最后一个元素（索引 N/2）的**实部**，其虚部恒为0。

(2) 交错实数数组（你的场景）

- 长度：N + 2 的 float（因复数需实部+虚部交错存储）
- **Nyquist存放规则**：
  - **位置**：数组末尾的两个元素（索引 N 和 N+1）
  - **值**：
    - output[N] = Nyquist频率的**实部**（有效值）
    - output[N+1] = 0（虚部，固定为0）

2. 示例说明（N=1024点FFT）

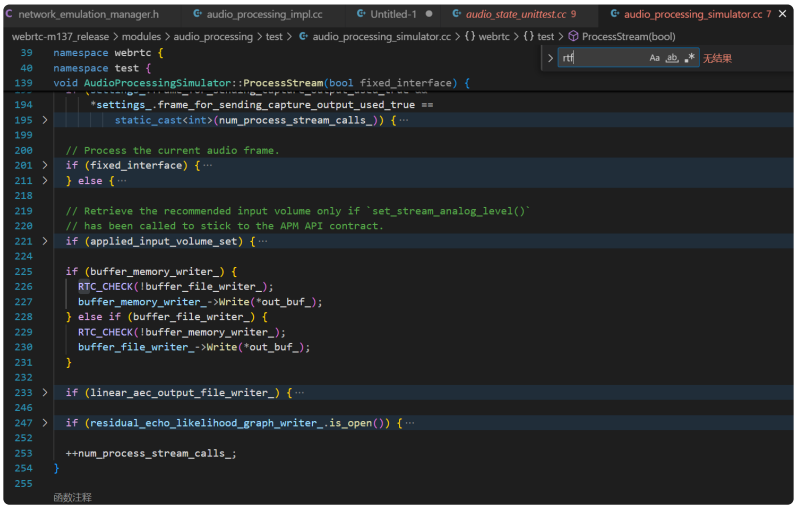
假设输入为 float input[1024]，输出为 float output[1026]（1024/2 + 1 = 513 个复数 → 需要 513*2 = 1026 个 float）：

索引范围	含义	Nyquist位置
output[0]	直流分量（实部）	-
output[1]	直流分量（虚部，恒为0）	-
output[2..1023]	正频率复数（实虚交错存储）	-
output[1024]	**Nyquist实部**	← 有效数据
output[1025]	**Nyquist虚部**（恒为0）	← 固定填充0

67. DC 分量是什么？ 答：信号的直流偏置。即信号的平均值，信号中不随时间变化的那一部分分量，即 fft 结果的 0Hz 部分，对应 freqdata[0]
68. 是在 王昶的 提交 43430adae18417cf118171e3faff902135f51e40 基础上做的，检查下有没有进 master ？
    a. 答：尚未合入，并且它在 master 分支上。 需要手动拉一下 。 http://gerrit-ce.tp-link.com/c/algorithm/video/nvmp_docker/+/312395
    b. 
    c. 不用太担心 ”mult_array“ 没有带来时间加速。一定是有加速的，这是成熟方案。
69. 问：

rtf 这个工作的难点，究竟是计时的实时性？ 还是如何统计帧耗时的 平均值、P95、P99；各模块 RTF、屏蔽 I/O 时间（直接 clock() 计时，但着重于给出详细统计数据和统计报告）？
看起来，如果要封成 rtf 模块的话，是需要 per-frame 更新和统计数据的吧

a. 答："没有太难的点。实时性用 C++ 的 choron， 统计报告也不涉及太难算的东西。"
b. "只需要统计 平均值、min、max，但是需要实时统计并展示。"
c. "不需要考虑 I/O ，只要你统计的时间点是对的，就不会计算 I/O"
d. "per-frame 更新，具体的，per-audio-frame 更新"
e. "不确定之前看到的那个 rtf 模块是不是 webRTC 源码的，可以在 webRTC 里面搜索 aec，然后看 aec 调用 rtf 的过程。'印象里那个模块就叫 rtf'"
    i. 已经确认 ProcessStream 函数就是处理 aec 的过程，但并没有搜到 rtf。 对源码的调研就到这儿吧
    ii.



f. 预期按照 **自适应滤波、aec、降噪、fft** 这几个模块分别统计。 预计 5 audio_frame 统计一次，这个在行业内没有太统一的标准

70. rtf 写 buffer_ring 来提高并发能力纯属脱裤子放屁，原本就是存个队列的事情，写了 buffer_ring 还要考虑 buffer_ring 失灵丢数据。太zhizhang了
71. RTF 常见坑
    a. 把音频I/O/文件读写计进来了 → 误判过慢。
    b. 用 CPU 时间而不是 wall-clock → 多线程/GPU 时不准确。
    c. 帧长不一致

1.
```
void mult_array(float *array1, float *array2, float *out, int N)

void sub_array(float *array1, float *array2, float *out, int N)

void fft() -> N=256~512
```

2. 仿真环境下写 prototype， 单元测试 512 点数的 向量 * + -. 小心循环 ÷ ; 检验 NEON 的加减法指令是分开的。（设备=arm v7）
3. fft Ne10；可能需要调研其他 fft 方法
4. 尽早上机。 机型=C220
5. 仿真环境 → compile.py & main.c（必定），可能要改 makefile

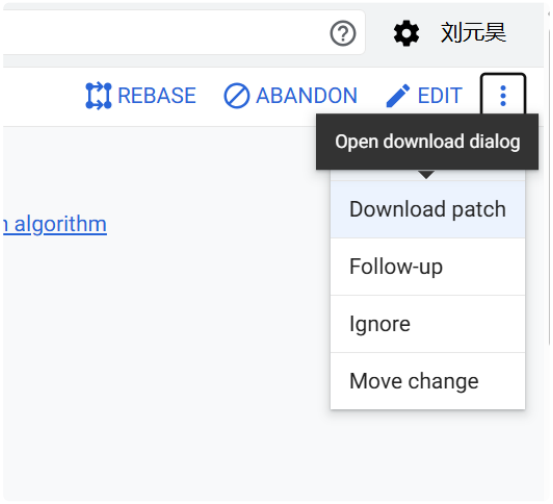
1. 可能的坑点："如果给出的参数允许，编译器会将某些循环展开之后生成SIMD目标代码……我在mmx、sse系列选项全部打开的gcc的行为中观察到过。"
2. 走读已有的 AUDIO_NEON 代码发现，涉及了相当多种类的 NEON 指令，包括但不限于：【看起来，基本是全都能用】

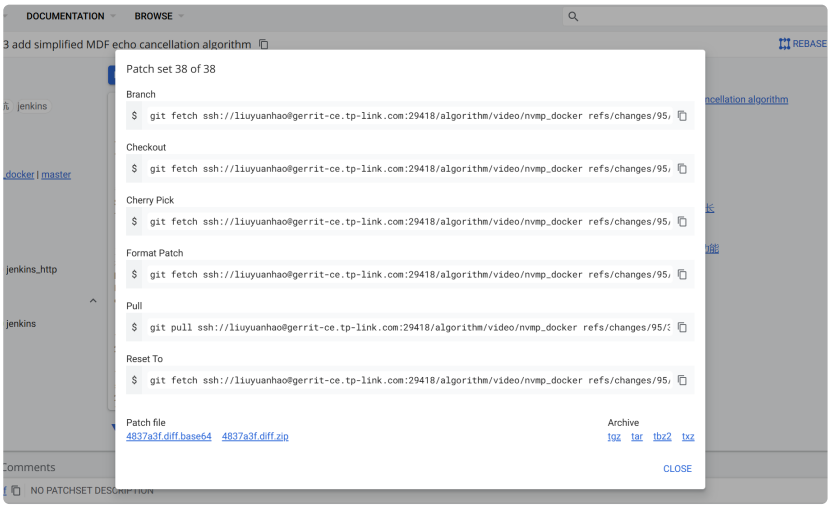
```
int16x4_t
int32x4_t
float32x4_t


vld1_s16
vmovl_s16
vdupq_n_s32
vmulq_s32
vsubq_s32
vshlq_s32
vaddq_s32
vshrq_n_s32
vmovn_s32
vabs_s16
vpmax_s16
vget_lane_s16

vmulq_f32
vrecpeq_f32
vsqrtq_f32
vdupq_n_u32
vceqq_u32
vmvnq_u32
vandq_u32
vreinterpretq_u32_f32
vrsqrtsq_f32
```

3. neon_boosted = nb 缩写
4. 在 gerrit 页面点击右上角查看对应的代码命令

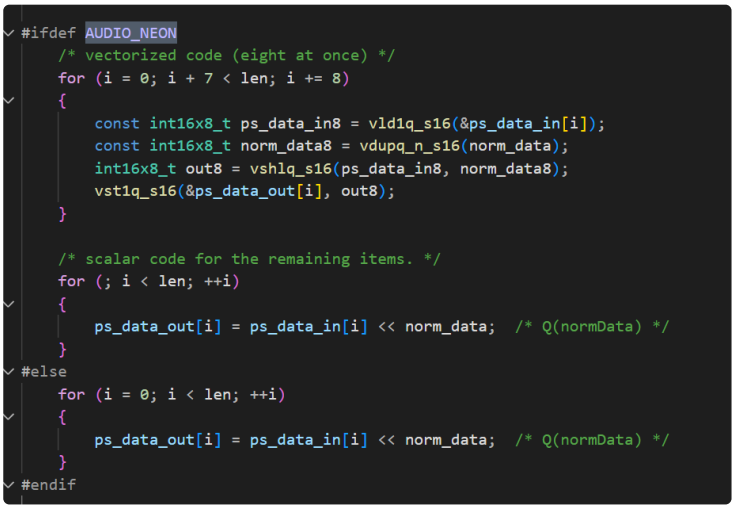
5. 



6. “Externally” 的意思是 会被外部参数更新

7. 

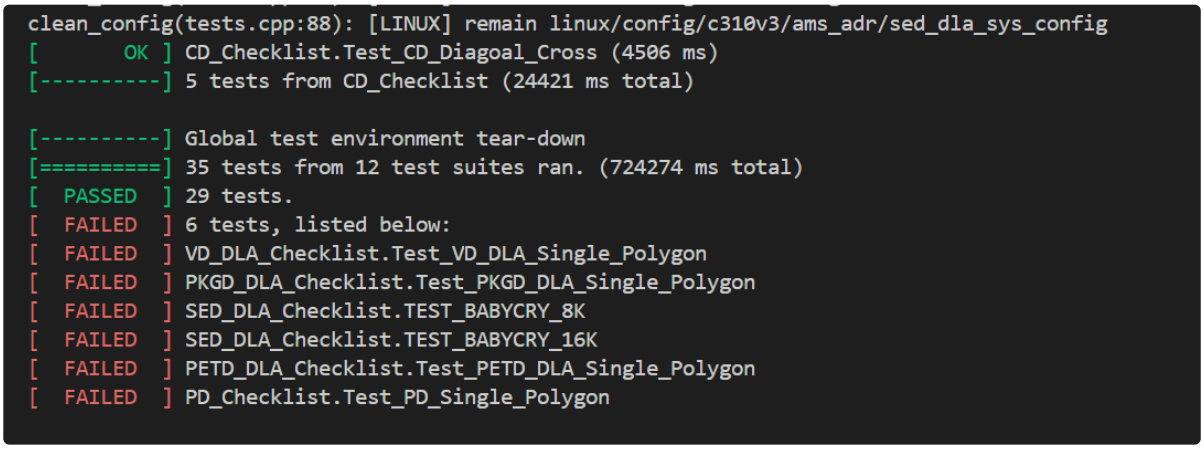
8. 搜索 AMS 中 AUDIO_NEON，对应开启 NEON 优化的宏定义。一块优化形如下图

9. 

```
#ifdef AUDIO_NEON
    /* vectorized code (eight at once) */
    for (i = 0; i + 7 < len; i += 8)
    {
        const int16x8_t ps_data_in8 = vld1q_s16(&ps_data_in[i]);
        const int16x8_t norm_data8 = vdupq_n_s16(norm_data);
        int16x8_t out8 = vshlq_s16(ps_data_in8, norm_data8);
        vst1q_s16(&ps_data_out[i], out8);
    }

    /* scalar code for the remaining items. */
    for (; i < len; ++i)
    {
        ps_data_out[i] = ps_data_in[i] << norm_data;  /* Q(normData) */
    }
#else
    for (i = 0; i < len; ++i)
    {
        ps_data_out[i] = ps_data_in[i] << norm_data;  /* Q(normData) */
    }
#endif
```

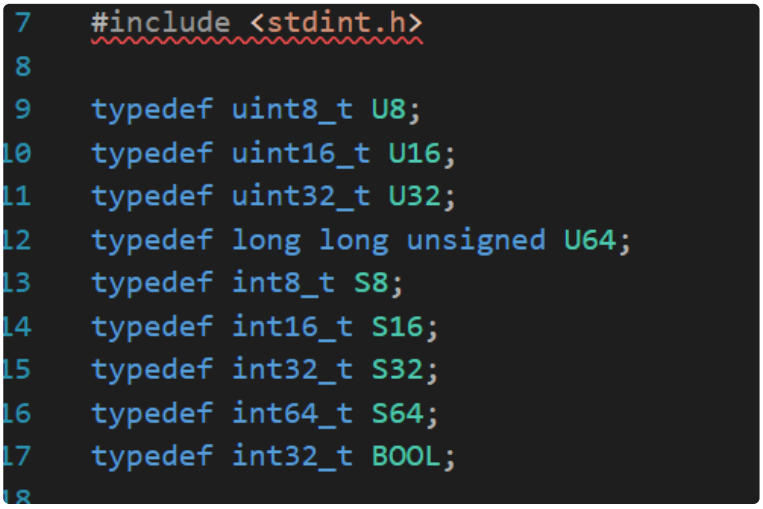
这感觉不就是 8 个 int16 压位到一个变量里面吗？

10. 

```
clean_config(tests.cpp:88): [LINUX] remain linux/config/c310v3/ams_adr/sed_dla_sys_config
[       OK ] CD_Checklist.Test_CD_Diagoal_Cross (4506 ms)
[----------] 5 tests from CD_Checklist (24421 ms total)

[----------] Global test environment tear-down
[==========] 35 tests from 12 test suites ran. (724274 ms total)
[  PASSED  ] 29 tests.
[  FAILED  ] 6 tests, listed below:
[  FAILED  ] VD_DLA_Checklist.Test_VD_DLA_Single_Polygon
[  FAILED  ] PKGD_DLA_Checklist.Test_PKGD_DLA_Single_Polygon
[  FAILED  ] SED_DLA_Checklist.TEST_BABYCRY_8K
[  FAILED  ] SED_DLA_Checklist.TEST_BABYCRY_16K
[  FAILED  ] PETD_DLA_Checklist.Test_PETD_DLA_Single_Polygon
[  FAILED  ] PD_Checklist.Test_PD_Single_Polygon
```

未改动代码的运行结果，29 tests passed。 来自 ./compile.sh -t

11. 

```
#include <stdint.h>

typedef uint8_t U8;
typedef uint16_t U16;
typedef uint32_t U32;
typedef long long unsigned U64;
typedef int8_t S8;
typedef int16_t S16;
typedef int32_t S32;
typedef int64_t S64;
typedef int32_t BOOL;
```

这里 S16= signed 有符号整数，U16 = unsigned 无符号整数

第 5 章　NEON 和 VFP 编程

5.1　扩展寄存器组 5-7
5.2　条件代码 5-9
5.3　一般信息 5-11
5.4　NEON 和 VFP 共享的指令 5-18
5.5　NEON 逻辑运算和比较运算 5-25
5.6　NEON 通用数据处理指令 5-33
5.7　NEON 移位指令 5-44
5.8　NEON 通用算术指令 5-50
5.9　NEON 乘法指令 5-63
5.10　NEON 加载 / 存储元素和结构指令 5-68
5.11　NEON 和 VFP 伪指令 5-76
5.12　NEON 和 VFP 系统寄存器 5-82
5.13　清零模式 5-86
5.14　VFP 指令 5-88
5.15　VFP 向量模式 5-97

12. 见 ARMV7+NEON汇编指令详解中文版.pdf ARMV7+NEON汇编指令详解中文版.pdf[4]

13. **饱和**
当运算结果超过目标数据类型的最大值或最小值时，结果会被钳制（clamp）到该类型能表示的最大或最小值，而非直接溢出（wrap around）

    示例：对int16_t类型（范围-32768~32767），若运算结果为40000，饱和后会输出32767；若为-40000，则输出-32768。
    应用场景：避免溢出导致的非预期行为，常见于信号处理、图像处理等需要数值稳定性的领域。

14. 

NEON 在机器翻译中被称为 “霓虹灯”.

15. 使用 arch 命令查看虚拟机架构。 虚拟机架构为 x86。使用systeminfo查看windows架构，使用x64【x64-based PC】

16. 杨一帆之前做的neon 2 sse 仿真，供参考： http://gerrit-ce.tp-link.com/c/algorithm/video/nvmp_docker/+/322286/1/ams/pc_dbug/linux/libdla/Makefile#28

4 https://pdconfluence.tp-link.com/download/attachments/157614785/ARMV7%2BNEON%E6%B1%87%E7%BC%96%E6%8C%87%E4%BB%A4%E8%AF%A6%E8%A7%A3%E4%B8%AD%E6%96%87%E7%89%88.pdf?api=v2&modificationDate=1756867171430&version=2

17.
```
72      table[i] = 7;
73      uint8_t indices[8] = {1, 3, 5.00, uint(-2), 9, 11, 31, 255};  // 测试越界索引
74      uint8_t tbl_out[8];

问题 4   输出   调试控制台   终端   端口   GITLENS

tbl_out[i]=7 7 7 7 7 7 7 0
tplink@tplink-vm:~/audio_demos/nvmp_docker/ams/src/lyh$
tplink@tplink-vm:~/audio_demos/nvmp_docker/ams/src/lyh$ g++ a.cpp -o a -I/home/tplink/audio_demos/nvmp_docker
out[i]=6 8 10 12
vtable[0]=7 7 7 7 7 7 7 7
vtable[1]=7 7 7 7 7 7 7 7
vtable[2]=7 7 7 7 7 7 7 7
vtable[3]=7 7 7 7 7 7 7 7
indices[i]={1}, out[i]={7}
tbl_out[i]=7 7 7 0 7 7 7 0
```
【错误示范】这里 indices 初始化中把 -2 和 5.00 赋值进来会引发 [-Wnarrowing] 即 warning 引入的报错，使用 "-w" 可以关掉。但出于尊重代码，应当使用 -Wall。

18. 报错？
```
utils.c: In function ‘resize_bilinear_c4’:
utils.c:1533:13: error: unknown register name ‘q12’ in ‘asm’
 1533 |             asm volatile(
      |             ^~~
utils.c:1533:13: error: unknown register name ‘q11’ in ‘asm’
utils.c:1533:13: error: unknown register name ‘q10’ in ‘asm’
utils.c:1533:13: error: unknown register name ‘q9’ in ‘asm’
utils.c:1533:13: error: unknown register name ‘q8’ in ‘asm’
utils.c:1533:13: error: unknown register name ‘q3’ in ‘asm’
utils.c:1533:13: error: unknown register name ‘q2’ in ‘asm’
utils.c:1533:13: error: unknown register name ‘q1’ in ‘asm’
utils.c:1533:13: error: unknown register name ‘q0’ in ‘asm’
utils.c:1533:13: error: unknown register name ‘r4’ in ‘asm’
make[2]: *** [<builtin>: utils.o] Error 1
make[2]: Leaving directory '/home/tplink/audio_demos/nvmp_docker/ams/pc_dbug/build_dir/linux/libdla'

make[1]: *** [Makefile:52: build_libdla] Error 2
make[1]: Leaving directory '/home/tplink/audio_demos/nvmp_docker/ams/pc_dbug/build_dir/linux'

make: *** [Makefile:272: build_linux] Error 2
build failed: 2
```
看起来是因为 #define _ENABLE_NEON导致的，实际在 pc_dbug 仿真环境默认是 #undef _ENABLE_NEON. 导致编译了一块汇编代码，这段代码只在 arm 上支持。

19. **关于 ./compile 和 ./main**

在pc_dbug目录下编译，传统算法与DLA算法需要分别编译测试。

(1) 传统算法编译

```
./compile.sh -t -d
```

传统算法测试

```
./main 0 --gtest_filter=*D_Checklist*
```

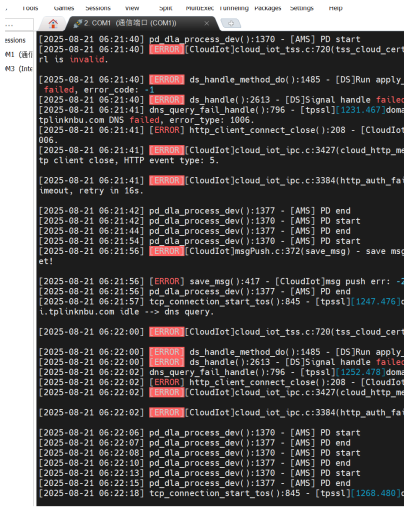
(2) DLA算法编译

```
./compile.sh -t -d --dla
```

传统算法测试

20.
```
./main 0 --gtest_filter=*DLA_Checklist*
```

21. 这次NEON优化应该主要涉及传统算法？ 深度学习需要调库的部分大 既率做不了 NEON



22. 重启串口，检查串口方向，重启 MobaXTerm，原理上讲是无需等待初始化的。
23.

5 刷机刷固件笔记

1. 编译固件 OnePage

2.
```
进入 NVMP 目录（而不是ams目录）
# cd ~/audio_demos/NVMPs/c220v2/NVMP

 切换到远程常电机分支  develop_sz0519（一般情况下，若电池机则bc_develop_220927分支，常电机则develop_sz0519分支）
# git ...  （直接vscode切换）
拉取所有代码，机型 -p c220v2 。 -n 意思是不编译

# ./scripts/build_main.sh -p c220v2 -s all -n
......
Current branch develop_sz0519 is up to date.
update product_config done
......update wlan module done
......update sdk done
ams: updating ams source ...
error: cannot pull with rebase: You have unstaged changes.
error: please commit or stash them.
这个符合预期，因为 ams 里有未commit改动，我也不希望它拉取

在 docker 的 attach shell 后，在 docker 内补充环境
不知道为什么，但以下命令需要逐行运行，-y是自动确认
# sudo apt-get update
sudo apt-get install -y bison
sudo apt-get install -y kmod
sudo apt-get install -y qt5-default
sudo apt-get install -y ccache
sudo apt-get install -y libgtest-dev
sudo apt-get install -y libopencv-dev
sudo apt-get install -y libmbedtls-dev
sudo apt-get install -y libunwind-dev
移除冲突的库
# sudo apt autoremove

切换到正确目录！
在 docker 清理上次的编译产物，防止权限问题
# cd /nvmp_source_dir/nvmp
# make clean
显示结果
 make[1] clean
 make[2] target/linux/clean
 make[3] -C target/linux clean

# make PR_NAME=c225v2 V=s -j1
```

```
卡在 if [ -s "/nvmp_source_dir/nvmp/build_dir/target-arm-openwrt-linux-gnueabi-c220v2/linux-ssc37x_ssc375/linux-5.10.117/patches/series" ]; then (cd "/nvmp_source_dir/nvmp/build_dir/target-arm-openwrt-linux-gnueabi-c220v2/linux-ssc37x_ssc375/linux-5.10.117"; if quilt --quiltrc=- next >/dev/null 2>&1; then quilt --quiltrc=- push -a; else quilt --quiltrc=- top >/dev/null 2>&1; fi ); fi
touch "/nvmp_source_dir/nvmp/build_dir/target-arm-openwrt-linux-gnueabi-c220v2/linux-ssc37x_ssc375/linux-5.10.117/.quilt_checked"
if [ ! -e /nvmp_source_dir/nvmp/build_dir/target-arm-openwrt-linux-gnueabi-c220v2/linux-ssc37x_ssc375/linux-5.10.117/Makefile ];then rm -rf /nvmp_source_dir/nvmp/build_dir/target-arm-openwrt-linux-gnueabi-c220v2/linux-ssc37x_ssc375/linux-5.10.117/* && cp -fpR "/nvmp_source_dir/nvmp/../sdk/soc/ssc37x/linux-5.10.117"/. /nvmp_source_dir/nvmp/build_dir/target-arm-openwrt-linux-gnueabi-c220v2/linux-ssc37x_ssc375/linux-5.10.117/; fi
卡住不动了？答：是正常的。 可以通过右侧看当前进程id

查询进程占用 和进程状态
# top -p <PID>

编译失败了，看起来是有人在别的模块代码里下毒了。
# 访问 \\exfile.tp-link.com\联洲国际研发\public\外销Camera提测\TapoCam\提测-勿删 的对应机型，查看最新一次编译的 “20250815.compile_commit_id”， 形如
[nsd]nvmp_bc_release_1.8.1:d3d02f0
# python3 分支节点切换.py /home/tplink/audio_demos/NVMPs/c220v2/NVMP /home/tplink/audio_demos/NVMPs/c220v2/compile_commit_id.txt

原理上讲 make 失败后，不 make clean 就可以执行下一次编译。 这样旧的编译.o结果自动不重复编译（如果源文件没修改）。
再次
# make PR_NAME=c220v2 V=s -j1

至此看到 Tapo_C225v2_en_1.1.2_Build_250916_Rel.7048n-dbg_up_boot.bin即为编译完成，其中 250911 为当日日期，dbg 表示开启串口板，7048是编译编号，en表示语言版本，Rel表示release，up_boot 表示 upgrade_bootloader。

将 up_boot 版本使用签名工具签名，目录=/home/tplink/audio_demos/NVMPs/c220v2/签名工具/mvsign.sh ， 依赖文件 signfwCE_new.x
sudo bash mvsign.sh /home/tplink/audio_demos/NVMPs/c225v2-neon/NVMP/nvmp/bin/ssc37x-c225v2/Tapo_C225v2_en_1.1.2_Build_250916_Rel.7048n-dbg_up_boot.bin
```

3. 串口板命令

```
# 密码：slpmstar
dbg_cmd reset
dbg_cmd get_sysinfo 1 # 获取mac
dbg_cmd onboarding llt-tp meiwenni
ubus call onboarding connect '{"ssid":" AZTE_T3000_BC07_5G ", "password":"87654321", "commit":1}'

#
dbg_cmd factory_write off
```

```
dbg_cmd get_cfg /tp_manage
dbg_cmd get_cfg /device_info
dbg_cmd set_sysinfo 1 00-FF-00-5C-1B-1F
dbg_cmd set_sysinfo 3 80215122C6A714B16B8CF0815C1B00EE23DD6D7A

# 常电模式
ubus call mcu dbg_set_power_mode '{"enable":1,"mode":1,"cap":100}'

# 压测CPU，可多次调用
yes > /dev/null &
# 查看CPU
top
# kill yes进程
pgrep yes  # 获取yes进程的PID
kill -9 PID  # 替换PID为实际的进程ID
# 修改日志等级
dbg_cmd dbg 0
dbg_cmd dbg 3
# 中断
killall -SIGBUS main

# 获取dump
tar -zcvf dump.tar.gz dump
tftp -p -l dump.tar.gz 192.168.18.2
tftp -p -l mic.pcm 192.168.18.2
tftp -p -l out.pcm 192.168.18.2
tftp -p -l audio_dump.pcm 192.168.18.2
tftp -p -l audio_dump_before.pcm 192.168.18.2
tftp -gr THD+sweep_16k.pcm 192.168.18.2

# ubus播放音频
cat>>default_alarm_config<<EOF
{"test":{"sample_rate":8000,"codec_type":2}}
EOF
ubus call speaker play_audio '{"file_name":"/lib/audio/siren_loop_16k.pcm",
"volume":"10"}'

ubus call speaker play_audio '{"file_name":"/lib/audio/sd_abnormal.g711",
"volume":"10"}'

# 修改播放参数
cat /etc/audio.config
sed -i 's/"max_ao_volume":-4,/"max_ao_volume":-1,/g' /etc/audio.config
cat /etc/webrtc_profile.ini
sed -i 's/set_compression_gain_db=15/set_compression_gain_db=11/g' /etc/
webrtc_profile.ini

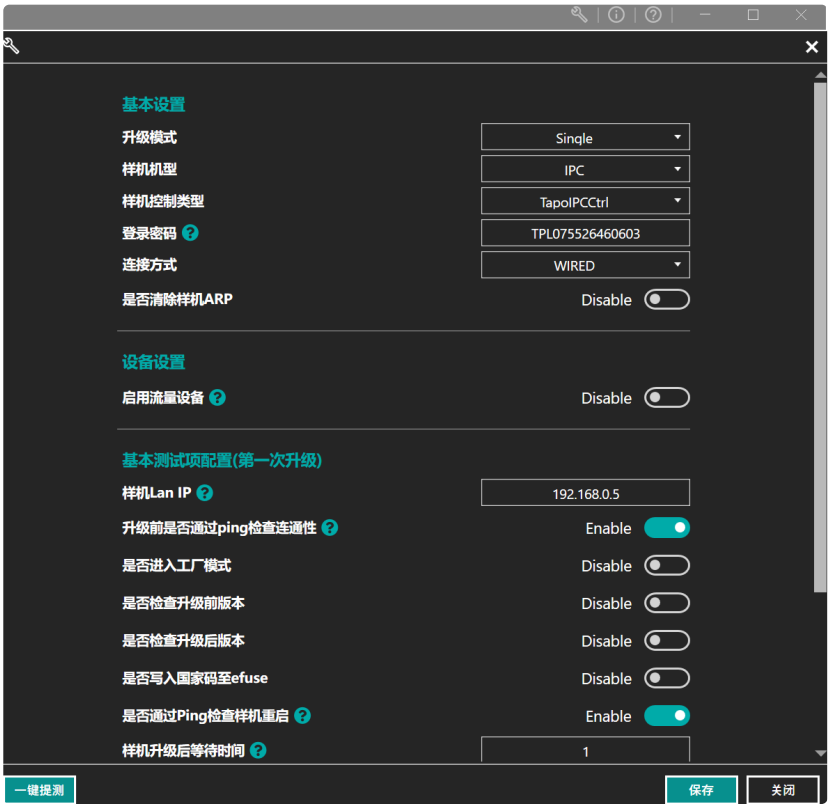
# 重启
killall -9 main
# 查看配置
dbg_cmd get_cfg /audio_capability
dbg_cmd get_cfg /audio_profile
```

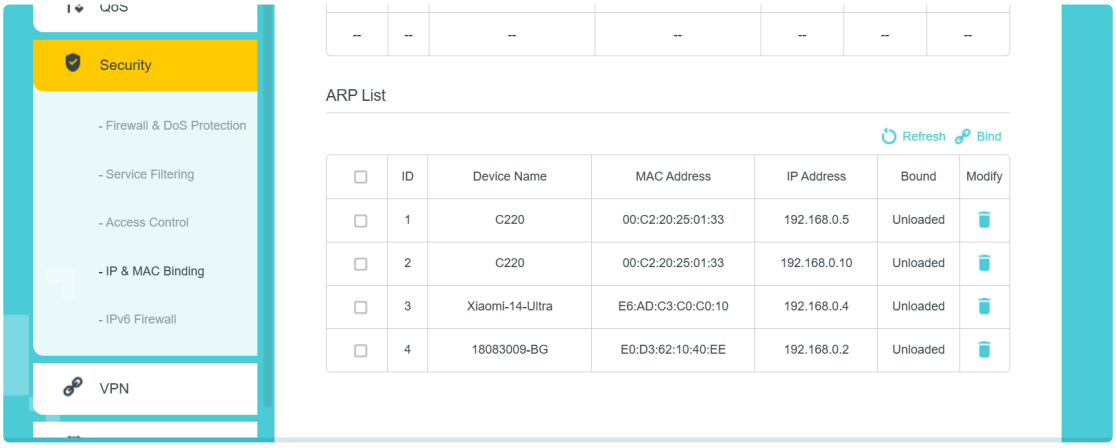
```
dbg_cmd get_cfg /device_info

cd /tmp/mnt/hard...
mkdir dump_ctl
du -sh * #看一下audio_dump_before.pcm
```

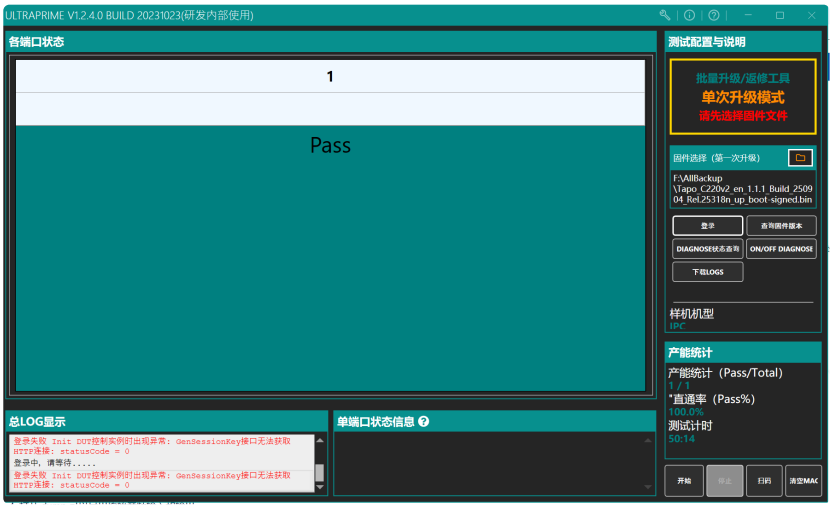
4. 刷固件流程：
    a. 拉仓库，编译前检查 config 的完备性
        i. /home/tplink/audio_demos/NVMPs/c220v2/NVMP/nvmp/ams_package/product_config/c220v2/buildams.config 的 CONFIG_AMS_AQI_LIST="aec3 ns3 tpaec3ns3"
        ii. /home/tplink/audio_demos/NVMPs/c220v2/NVMP/nvmp/product_config/c220v2/isp_config/C220_2.0/base-files/etc/audio.config 的 aec3_param 和 audio_factory_profile
        iii. /home/tplink/audio_demos/NVMPs/c220v2/NVMP/nvmp/product_config/c220v2/isp_config/C220_2.0/base-files/etc/system.config 的 device_info 版本号和机型名称
    b. 使用 ultraprime 和 mobaXTerm 连接摄像头。通过 mobaXTerm 连接串口板连接摄像头，账号=root，密码=slpmstar 。 使用 ultraprime 和路由器连接串口板，配置设备的 ip，可以在本地用 cmd ping 一下。 设备 ip 来自路由器设置页面。 配置如图所示。路由器设置页网址 = http://192.168.0.1/

c.





d. 使用 ultraprime 登录。点击“检查当前固件版本”。截图。然后选择要刷的固件，点击“开始” 开始通过路由器传输固件。在 MobaXTerm 检查串口板的输出有无报错。

e. 

f. 出现 pass 说明刷机成功，摄像头重新通过路由器连接到了电脑，ping 通了。点击 “查询固件版本” 查看当前固件版本。（一般版本号上有编译日期）

g. 使用手机 Tapo App 检查设备功能可用。串口板可能会被禁用。 至此刷机成功！

1. 打开 dump 可以导出原始音频输入和输出

   http://gerrit-ce.tp-link.com/c/algorithm/video/nvmp_docker/+/312441

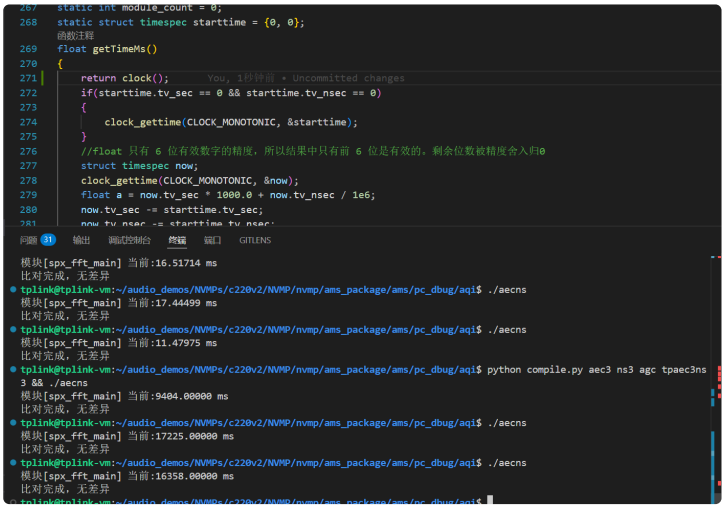
2. AMS_ERROR 等信息会打印到串口
3. 有些发布版会关闭串口输出，这时从串口看不到信息
   a. 如何启用串口板？
4. 串口命令 MobaXTerm
5.

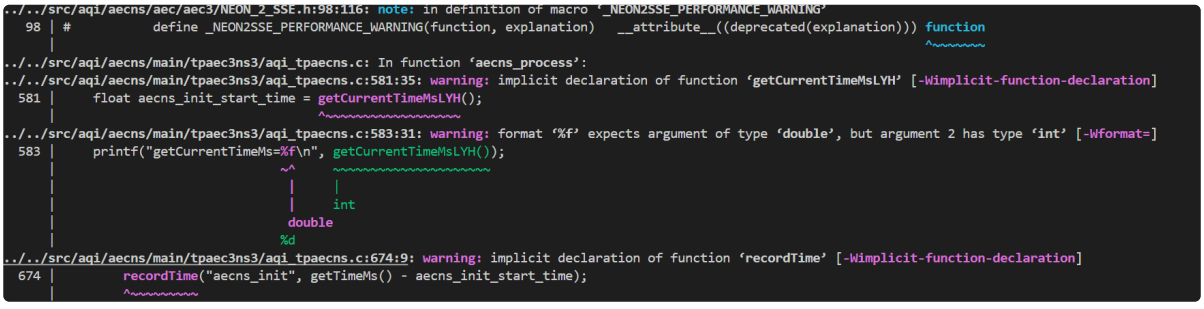
```
# 重置设备 = reset键
dbg_cmd reset

# 连接 2.4GHz Wifi
dbg_cmd onboarding 路由器名 密码
dbg_cmd onboarding llt-tp meiwenni
```

6. MobaXterm 连接串口并登录。 账号=root，密码=slpmstar

6 笔记

1. 最后解法：更换 1102046774 账号（fzy12345）并更换测试包。归因：测试包使用beta云，而正式包使用正式云。导致dbg固件访问beta云相关配置的时候挂了。不允许 app 访问流量网络≠关闭流量开关，这app好像会偷偷拿手机状态信息，在进入 app 的一瞬间，选择是否“local”模式
2. 为什么编译并运行， 会比直接运行快 10ms？？ 已经确认了不是 “CLOCK_MONOTONIC” 高精度时钟带来的，也不是 freopen 带来的
3. 

4. 如果 Ctrl+S 和 编译按得太快了，会导致文件还没保存到虚拟机，导致编译的源文件不是最新版。
5. C 语言中函数的隐式声明：留个坑。被搞了个奇怪的bug。 **没有在当前文件显式 #include "aec3_utils.h"** ，导致编译器在编译阶段，默认给了 getCurrentTimeMsLYH() 一个预定义（虽然我不知道为什么要给）。
然后链接到 aec3_utils.c 的时候，又访问到了对应的函数，导致 debug 的时候可以进入正确的函数。
但 return 的返回值 并没有链接到 正确的位置
6. 

7. 通过 -save-temps 和 查看汇编结果 .s 文件，可以搜索你的函数名
warning: implicit declaration of function ‘getCurrentTimeMsLYH’ [-Wimplicit-function-declaration]

8. 宏定义位置 作用域范围
   .h 文件 所有包含该头文件的 .c 文件
   .c 文件 仅当前 .c 文件及其包含的头文件 其他 .c 文件无法直接访问（除非通过 extern 声明，但宏不支持此操作）。

9. static 限制作用域：变量仅在定义它的源文件内可见，其他文件无法通过 extern 引用。
10. 关于 CLOCK_REALTIME 和 CLOCK_MONOTONIC 【Linux】

    CLOCK_REALTIME表示机器对当前挂钟、一天中的时间的最佳猜测。这意味着CLOCK_REALTIME可以随着系统时间时钟的改变而向前和向后跳转。

    CLOCK_MONOTONIC表示自过去某个任意固定点以来经过的绝对挂钟时间。它不受系统时间时钟变化的影响。

11. 推荐使用 printf 调试，确认当前部分被编译+运行了；推荐每次运行前删除上一次产物
12.

```
    }
    // printf("我真的运行到了\n"); // debug
    // for(i = 0; i < N; ++i)
    // {
    //     out[i] = g_tmp_out[i]; // 防止被 -O3 优化掉 C 的运算结果
    // }       You, 15秒钟前 • Uncommitted changes
```

13. 我服了，因为 array1 和 out 的地址会重复，所以我单元测试写错了，导致报错调试了半天。neon本身的精度没问题。
14.

```
    函数注释
168 void calc_array(char symbol, float *array1, float *array2, float *out, int N) // 注意 array1 和 out 会重复地址!
169 {
170     if(symbol != '+' && symbol != '-' && symbol != '*')
171     {
172         printf("error SYMBOL: %c\n", symbol);
173         return ;
174     }
175     int i;
176     int loop_end = N^(N&3); //=N-N%4
177
178
179
180         for(i = 0; i < N; ++i)
```

15. ams/aqi 模块的 master 分支被下毒了，如果这时候用 cherrypick 去拉其他变更，会被这个下毒卡住
16.

```
tplink@tplink-vm:~/audio_demos/NVMPs/c220v2/NVMP/nvmp/ams_package/ams/pc_dbug/aqi$ python compile.py aec3 ns3 agc tpaec3ns3 && ./aecns 7 input/gf_mic.wa
v  input/gf_spk.wav  input/gf_aec_state.wav  input/gf_motor_state.wav
../../src/aqi/aecns/aec/aec3/audio_aec.c: In function 'speex_EXpara_apply':
../../src/aqi/aecns/aec/aec3/audio_aec.c:605:79: error: 'AEC3_SYS_CONFIG' {aka 'struct _AEC3_SYS_CONFIG'} has no member named 'freq_bands_table'
  605 |             p_context->den->fft_eq->freq_bands_table[i] =  (float)EX_ctrl_para->freq_bands_table[i];
      |                                                                               ^~
../../src/aqi/aecns/aec/aec3/audio_aec.c:606:79: error: 'AEC3_SYS_CONFIG' {aka 'struct _AEC3_SYS_CONFIG'} has no member named 'gain_bands_table'
  606 |             p_context->den->fft_eq->gain_bands_table[i] =  (float)EX_ctrl_para->gain_bands_table[i];
      |                                                                               ^~
../../src/aqi/aecns/aec/aec3/audio_aec.c:609:73: error: 'AEC3_SYS_CONFIG' {aka 'struct _AEC3_SYS_CONFIG'} has no member named 'motor_gain_db'
  609 |         p_context->den->motor_gain_linear = exp(DB_LINEAR * EX_ctrl_para->motor_gain_db);
      |                                                                         ^~
bash: ./aecns: No such file or directory
```

17. 什么是 Gerrit
    a.

Gerrit 特性详解

1. 代码必须经过审查才能合入主分支
2. 所有操作有审查痕迹
3. 分支保护

Change Gerrit 中的一个“改动”，对应你推送到 refs/for/<branch> 的提交。一个 Change 可以包含多个 Patch Set。
Patch Set Change 的一次提交或修改。每次修改后更新 Patch Set，保留历史。
Reviewer 审查者，可以评论和给出评分（+1、+2、-1 等）。
Label 评分类型，常用的有 Code-Review、Verified 等，用于决定 Change 是否可以合入。
Cherry Pick 在 Gerrit 中可以把 Change 应用到其他分支生成新 Change。
Topic 一个可选字段，用于把多个相关 Change 组织在一起。

3.1 代码审查（Code Review）
每次 push 到 refs/for/<branch> 都生成一个 Change。
3.3 提交历史管理
Gerrit 强制所有提交必须有 Change-Id（可通过 commit-msg hook 自动生成）。
使得每次提交都可以追踪到 Gerrit Change。
提交历史干净，避免直接修改 master 的提交历史。
3.4 Patch Set 更新
如果审查者要求修改代码：
1. 在本地修改代码
2. git commit --amend 更新提交（保持 Change-Id 不变）
3. git push origin HEAD:refs/for/<branch>
Gerrit 会在原 Change 下生成新的 Patch Set，而不是新建 Change。
3.6 集成 CI/CD
Gerrit 可以集成 CI 系统（如 Jenkins、Zuul）：
每次 Change 推送触发自动构建和测试。
测试通过才能批准 Change（Verified 标签 +1/+2）。
3.8 命令行集成
常用命令：
git fetch <repo> refs/changes/xx/ChangeNumber/patchset
git push origin HEAD:refs/for/<branch>（提交新 Change）
git review（可用 review 工具简化操作）

因此，普通的 git push origin master 会被拒绝，因为 Gerrit 不允许直接改写 master。
refs/for/master 的作用
在 Gerrit 中，你推送到：
refs/for/<branch>

例如：
git push origin HEAD:refs/for/master
意思是：
“我想把当前分支的提交推送到 master 分支，但先作为待审查的改动（Change）提交到 Gerrit。”
> 关键点：refs/for/<branch> 是 Gerrit 的特殊命名，用于触发 Code Review 流程，而不是直接写入分支。

18. 什么是 Far Frame？AEC（Acoustic Echo Cancellation，声学回声消除） 时，为什么需要引入 Far Frame（远端帧） 这个概念？
    a. 远端信号（Far-end）：对方说话的声音，从扬声器播出。在 WebRTC Audio Processing Module (APM) 里，Far Frame = Reverse Stream：

    b. 近端信号（Near-end）：本地麦克风采到的声音

    c. AEC 的处理流程一般是这样的：
       1. 拿到 Far Frame（远端 10ms PCM 块）作为自适应滤波器的输入参考。这是被扬声器播放的内容。
       2. 拿到 Near Frame（近端 10ms PCM 块）这是麦克风录下来的内容，含本地语音 + 回声。

19. 音频仿真命令

20.
```
pc_dbug 前需要打开仿真开关，修改
1. ams_common.h
#define PCDBUG_AEC_ENABLE
#define PCDBUG_AEC_STATE_FILE_ENABLE
#define PCDBUG_MOTOR_STATE_FILE_ENABLE
2.



# 编译
python compile.py aec3 ns3 agc tpaec3ns3

# 相对路径的运行
./aecns 7 input/gf_mic.wav  input/gf_spk.wav  input/gf_aec_state.wav  input/
gf_motor_state.wav

# 音频放绝对路径，不放在 git 仓库内
./aecns 7 /home/tplink/audio_demos/audio_input/gf_mic.wav  /home/tplink/
audio_demos/audio_input/gf_spk.wav  /home/tplink/audio_demos/audio_input/
gf_aec_state.wav  /home/tplink/audio_demos/audio_input/gf_motor_state.wav



# 绝对路径的运行 输出文件在工作目录下
/home/tplink/audio_demos/nvmp_docker/ams/pc_dbug/aqi/aecns 7 /home/tplink/
audio_demos/nvmp_docker/ams/pc_dbug/aqi/input/gf_mic.wav /home/tplink/
audio_demos/nvmp_docker/ams/pc_dbug/aqi/input/gf_spk.wav /home/tplink/
audio_demos/nvmp_docker/ams/pc_dbug/aqi/input/gf_aec_state.wav  /home/tplink/
audio_demos/nvmp_docker/ams/pc_dbug/aqi/input/gf_motor_state.wav

# 编译并运行
cd /home/tplink/audio_demos/nvmp_docker/ams/pc_dbug/aqi
python compile.py aec3 ns3 agc tpaec3ns3 && ./aecns 7 input/gf_mic.wav  input/
gf_spk.wav  input/gf_aec_state.wav  input/gf_motor_state.wav
```

```
python compile.py aec3 ns3 agc tpaec3ns3 && ./aecns 7 /home/tplink/audio_demos/
audio_input/gf_mic.wav  /home/tplink/audio_demos/audio_input/gf_spk.wav  /home/
tplink/audio_demos/audio_input/gf_aec_state.wav  /home/tplink/audio_demos/
audio_input/gf_motor_state.wav
```

运行时报错
1. Floating point exception？
可能是当前的分支未联调带来的问题。 nvmp_release_1.8 的最新代码不一定能直接运行
/home/tplink/audio_demos/NVMPs/c220v2/NVMP/nvmp/ams_package/ams/pc_dbug/aqi/
aecns 7 /home/tplink/audio_demos/audio_input/gf_mic.wav /home/tplink/
audio_demos/audio_input/gf_spk.wav /home/tplink/audio_demos/audio_input/
gf_aec_state.wav /home/tplink/audio_demos/audio_input/gf_motor_state.wav
答： 把音频放到 input 以内就能正确执行了，怀疑是因为绝对路径和工作目录的环境变量不同，或者是某些静态库链接会有分歧。
最后发现很可能是“没找到文件”

21. 如何 push 代码到远程而保持不合入？

22. 如何单独 cherry pick 另外一个分支 dev-branch 上的 commit ，例如 commit-id=1547ce3d9？【仓库迁移，commit迁移，git remote】
答：
方法1：使用远程分支。
方法2：使用 git remote ，在本地的两个仓库直接建立连接。

```
# 在目标仓库里
cd /path/to/target-repo

# 添加源仓库
git remote add source-c225v2-neon /home/tplink/audio_demos/NVMPs/c225v2-neon/
NVMP/nvmp/ams_package
git fetch source-c225v2-neon

git remote add source-repo /path/to/source-repo th/to/source-repo
git fetch source-repo
# 示例
git remote add source-v220
git fetch source-v220
# 查询<commit-hash>是否拉过来了


# 挑选特定提交
git cherry-pick <commit-hash>
```

追加提交，它可以在不增加一个新的 commit 的情况下，将新修改的代码追加到前一次的commit 中。进入修改commit信息的位置直接 ctrl+x退出即可
```
#  git commit --amend
```
也可以 git commit --amend --no-edit ，其中--no-edit 表示不修改 commit message

回退若干个 commit ，可以隔空回退，左开右闭区间
```
# git revert --no-commit <最新commit_id>..<最旧commit_id>
 git revert --no-commit 05fe54bc8..ca3d2a2c3
```

把远程代码拉下来，把自己的commit放在最后面
```
# git rebase
First, rewinding head to replay your work on top of it...
Applying: WIP：迁移neon更改

# git status
On branch nvmp_release_1.8
Your branch is ahead of 'origin/nvmp_release_1.8' by 1 commit.
```

提交所有 commit
```
# git push origin HEAD:refs/for/nvmp_release_1.8
Compressing objects: 100% (27/27), done.
remote: SUCCESS
remote:   http://gerrit-ce.tp-link.com/c/algorithm/video/nvmp_docker/+/329791
WIP：迁移neon更改 [NEW]
To ssh://gerrit-ce.tp-link.com:29418/algorithm/video/nvmp_docker
 * [new branch]          HEAD -> refs/for/nvmp_release_1.8
```

单独提交 commit：先 cherry-pick 将对应 commit 从本地change分支，切到本地 master（或 nvmp_release_1.8）分支，然后使用 git push
先切换分支
```
# git checkout nvmp_release_1.8
```
删除无需提交的本地commit （这个分支上应该是干净的）
```
# git reset --hard HEAD~1 # 直接回退和删除最新的本地 commit，不保存到暂存
```
cp 本地的想提交的 commit
```
# git cherry-pick 5dd46860f
```
检查git stauts后 推送到远程
```
# git status
On branch nvmp_release_1.8
Your branch and 'origin/nvmp_release_1.8' have diverged,
and have 1 and 4 different commits each, respectively.
  (use "git pull" to merge the remote branch into yours)
# git push origin HEAD:refs/for/nvmp_release_1.8
```

如果报错，检查是否绑到远程
```
# git remote -v
origin  ssh://liuyuanhao@gerrit-ce.tp-link.com:29418/algorithm/video/
nvmp_docker (fetch)
origin  ssh://liuyuanhao@gerrit-ce.tp-link.com:29418/algorithm/video/
nvmp_docker (push)
```

把 commit 变成 stash

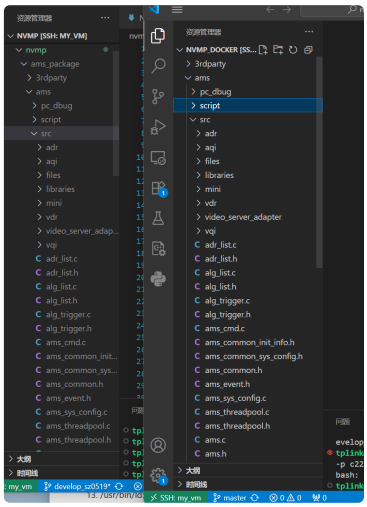
```
# git reset HEAD~1 --soft
# 撤销最近一次提交，保留修改到暂存区
  git stash
# 将暂存区的修改保存为 stash

只加入 modified 的内容，不加入 Untracked files:
# git add -u
```

23. 调试 debug 使用的 vscode 配置？ 注意修改 program 的绝对路径

24.

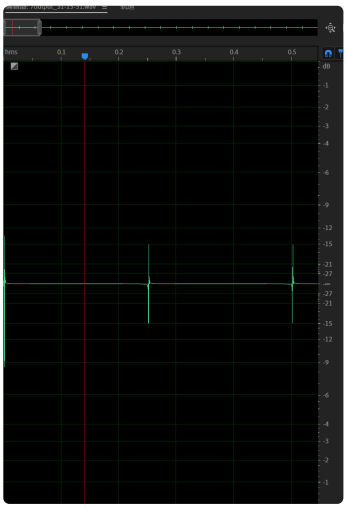
```
{
    // 使用 IntelliSense 了解相关属性。
    // 悬停以查看现有属性的描述。
    // 欲了解更多信息，请访问: https://go.microsoft.com/fwlink/?linkid=830387
    "version": "0.2.0",
    "configurations": [
        {
            "name": "调试aqi-pc_dbug",
            "type": "cppdbg",
            "request": "launch",
            "program": "/home/tplink/audio_demos/NVMPs/c220v2/NVMP/nvmp/
ams_package/ams/pc_dbug/aqi/aecns",
            "args": [
                "7",
                "/home/tplink/audio_demos/audio_input/gf_mic.wav",
                "/home/tplink/audio_demos/audio_input/gf_spk.wav",
                "/home/tplink/audio_demos/audio_input/gf_aec_state.wav",
                "/home/tplink/audio_demos/audio_input/gf_motor_state.wav"
            ],
            "stopAtEntry": false,
            "cwd": "${workspaceFolder}",
            "environment": [],
            "externalConsole": false,
            "MIMode": "gdb",
            "setupCommands": [
                {
                    "description": "为 gdb 启用整齐打印",
                    "text": "-enable-pretty-printing",
                    "ignoreFailures": true
                },
                {
                    "description": "将反汇编风格设置为 Intel",
                    "text": "-gdb-set disassembly-flavor intel",
                    "ignoreFailures": true
                }
            ]
        }
    ]
}
```

25. 如何连接 Tapo 设备？ 第一次使用 Tapo 设备，使用蓝牙自动连接失败后，需要手动配置 Wi-Fi 连接。手动连接时，需要先将手机连接到设备 Wi-Fi TAPO_CAM_XXXX ，这里后四位=设备MAC，在机身上有，或者看路由器。配置完网络密码后，再将手机连接至同一局域网 Wi-Fi。
    a. 卡在“正在连接您的网络”？将手机连接到路由器，等待。
    b. 使用Tapo App 的时候不要开了流量！
26. 在 Docker 容器中使用 apt-get 安装库时，不会直接安装到宿主机上。Docker 容器与宿主机是隔离的环境，具体原因和注意事项如下：
    a. Docker 容器通过 Linux 命名空间（Namespaces）和控制组（cgroups）实现资源隔离，容器内的文件系统、进程、网络等与宿主机完全独立
    b. 在容器内运行 apt-get install 时，库只会安装到容器的文件系统中（如 /usr/lib 或 /usr/local/lib），不会影响宿主机的文件系统
    c. 卷挂载（Volume Mount）:如果容器启动时通过 -v 参数将宿主机的目录挂载到容器内（例如 -v /宿主机路径:/容器路径），则在容器内安装的库可能会写入挂载的宿主机目录中。但这种情况需要显式配置，默认情况下不会发生。
27. logging.error ≠ logging.ERROR !! AMS_ERROR ≠ AMS_error 。AMS_ERROR 的日志会打印到串口，在本机上使用 printf 即可
28. 小心 vscode 的跳转，可能会从目前的 src 目录跳转到 build_dir/src 下的重名文件。
29. tapo设备=C250，室内云台机，目前正常。
30. AZTE_T3000_BC07 87-----1。
31. tapo设备=C260，室内云台机。现在卡在 tapo app 链接不上机器 ，冷却一下重新上电试试。怀疑wifi模块挂了。最后结论：Reset键损坏，只能通过串口Reset；云台无法控制，原因未知；连接到 app 时使用蓝牙自动连接失败，改用 局域网 WiFi 手动连接后，成功。
32. 几番辗转发现深色 vscode 更舒服，即使 light 模式下看字母更清晰。
33. http://gerrit-ce.tp-link.com/admin/repos/IPC/NVMP 是NVMP仓库，http://gerrit-ce.tp-link.com/admin/repos/algorithm/video/nvmp_docker 是音频算法的仓库。其中 NVMP_DOCKER 的内容就是 NVMP/ams_package 的内容。相关md文档 \\exfile.tp-link.com[5]\联洲国际研发\消费产品研发处\03-消费电子研发部\06-音视频技术课\public\音频相关\新人软件共享\文档
34. 

5 http://exfile.tp-link.com

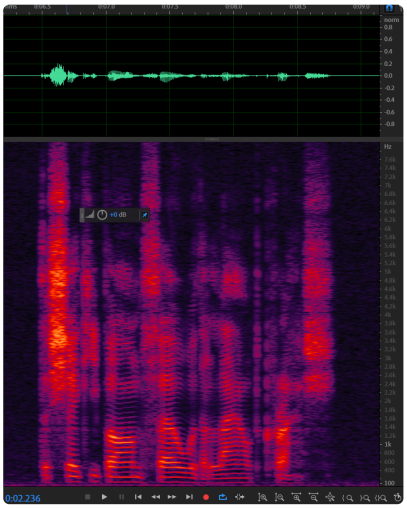
35. 总谐波失真（THD）：输入纯单音信号时，输出信号中谐波成分与基波的功率比，通常以百分比表示。THD<1%为高保真标准，105 dB SPL下THD>3%可能影响语音识别精度
36. 音频麦克风的线性性（Linearity）是指麦克风输出电信号与输入声压之间保持线性比例关系的特性，这是衡量麦克风信号保真度的重要指标。
37. DSP硬件 （数字信号处理器），专门为高效执行数字信号处理算法而设计的微处理器。
38. 裸流丢帧：是指在未封装容器格式（如H.264/H.265裸流）的**视频传输**过程中，丢弃部分视频帧。例如带宽不足时，通过丢帧降低码率，优先保留关键帧（I帧）。
39. 音量映射/响度映射：人耳感知的“响度”（Loudness）与物理声压级（dBSPL）并非线性关系，需通过等响曲线（Phon单位）或心理声学模型（Sone单位）进行映射，以匹配主观感受
40. Bug recalled 通常指因软件或硬件中存在的缺陷（bug）导致产品被**召回**或紧急修复的情况。
41. Bug assign（缺陷分配）指将已发现的软件缺陷（Bug）分配给特定的开发人员或团队进行修复的过程
42. 出货意见：对货物样品的修改要求
43. 循环index不要用 l（小写L、大写I ，这些字符容易和 1 混淆所以不用。 正常的 i,j,k 是可用的）。
44. **内存管理原则**：仅对 malloc/calloc/realloc 分配的内存调用 free，其他变量由系统自动处理。
45. /usr/bin/ld: /tmp/cczdgixI.o: in function `min':
audio_io.c:(.text+0x0): multiple definition of `min'; /tmp/ccdnV2eK.o:generate.c:(.text+0x0): first defined here?
    a. 这表示 min 被重复定义了，是数学库里自带了 fmin 导致的。 一般 max\min 应当用 #define 定义。
    b. “-lm” 要写在最后，链接数学库，和一般的编译选项不同。
46. 关于知识点的思维导图，我应该主要看哪一部分？
答：主要看3A算法（AGC+ANR+AEC）
47. 讨论时常提到的 16K 和 8K 是什么意思？是指采样率吗？
    a. 答：指 FFT 的点数M，其实也是采样率。16K和8K在算法上无差异，但调参上需要重新做。
48. PPT的问题。“Gain” vs “AGC”
    a. 答：Gain是固定增益，AGC是自适应增益，AGC会随音频的变化而变化
49. HS = 啸叫抑制
50. 原信号和**反相信号**和**时间反转信号**的**幅度谱**是一致的，但复数频谱仍然不同。通过希尔伯特变换构造的解析信号 也可能共享相同幅度谱。
51. 窗函数在信号处理中主要用于抑制**频谱泄漏**（Spectral Leakage）和优化频率分析精度。其核心原理是通过对时域信号两端进行平滑加权，减小截断信号的边界突变（吉布斯现象），从而在频域中降低旁瓣能量并调整主瓣特性
52. 根据奈奎斯特采样定理（Nyquist Sampling Theorem），采样频率必须**至少大于信号最高频率的两倍**，才能完整重建原始信号而不产生混叠（Aliasing）现象。混叠（Aliasing）现象中的频率映射过程**本质上是一种以采样频率为基准的模运算**.
53. **动态分配数组的大小无法通过 sizeof 获取**
    - malloc 分配的内存大小信息由 C 库内部管理（如 malloc_chunk 结构记录），但程序员无法直接通过 sizeof 获取

- sizeof 是编译时运算符，对于动态分配的内存，它只能看到指针类型，而非实际分配的空间

54. 无限多同幅度，同相位，随机频率的正弦波，叠加会得到时域上的冲激信号；对于离散情况，会形成周期的冲激信号，其中周期T = 所有离散信号的周期的最小公倍数；对于连续的情况，最小公倍数→+∞，所以就得到单一的冲激信号。
55. 
56. 无限多同幅度，随机相位，随机频率的正弦波，（均匀随机），理论上讲，叠加的结果是 0. 该结论等价于，对于任意一个组成成分的正弦波，必然存在一个它的反相。
57. 尝试在离散的情况下做实验，发现对于 2000个采样点，频率固定为5Hz，相位偏移在 Π和0 之间随机。原理上讲，只有正波和反相波一样多时，结果为单频率波形或。实际却还是得到了白噪音。不知为何？
58. 白噪音的一般生成方法是在时域直接随机。
59. 自然界的正弦波，一般认为相位是均匀随机，频率和幅度是服从高斯分布的随机。
60. 各态历经的随机过程
61. 语音用语谱图展示。

    用这里面的精简测试音源裁剪，注意copy到本地裁剪：\\exfile.tp-link.com[6]\联洲国际研发\消费产品研发处\03-消费电子研发部\06-音视频技术课\public\音频相关\音频评价方法\直播音频调试与评价

6 http://exfile.tp-link.com



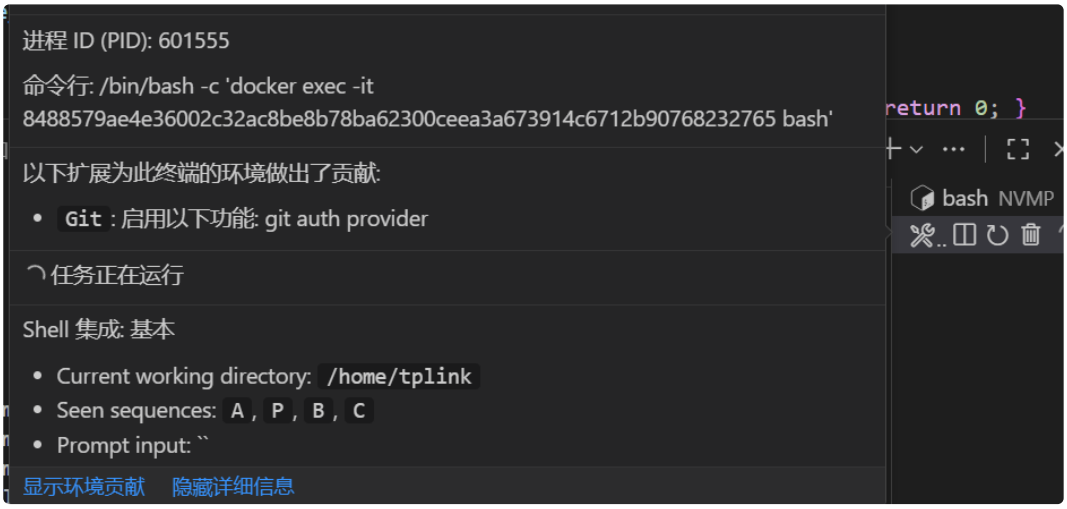
62. **C 语言规范**
    a. 函数声明放 .h 头文件，定义放 .c，这是防止一个函数的定义被多次链接到可执行文件里。声明和函数定义的关系 使用 makefile 处理。
    b. 函数名不要取 print 这种像自带函数的名字，可以print_wav，但你这里的功能应该是save_pcm（保存音频data块）
    c. y 的内存管理问题严重！Filter 函数里面开辟，直接return出去，这是绝对不允许的。 即在函数内 malloc 的变量，一定要在 return 前释放掉。或者更规范的做法是在调用前开辟空间，然后传一个 预期的结果指针进来。这样可以避免在函数内开辟空间。
    d. C语言几乎完全不支持给入参设置默认值。

会议关键词：车噪不清晰、直播场景、周期性噪音、门铃导致对讲不清晰、C-系列的摄像头、摄像头产品线分类（泛光）

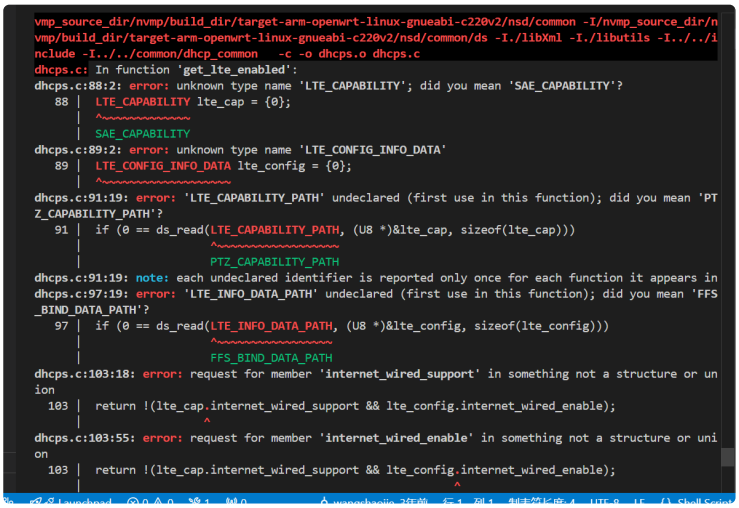


1. 直接复制目录到另一个仓库，会遇到脏东西！

2.
```
   e7ac8d8..48613d4  nvmp_bc_release -> origin/nvmp_bc_release
   6ec1da4..cfb62a0  nvmp_release_mercusys_250707_red -> origin/nvmp_release_mercusys_250707_red
Current branch nvmp_release_1.8 is up to date.
Already on 'nvmp_release_1.8'
Your branch is up-to-date with 'origin/nvmp_release_1.8'.
Current branch nvmp_release_1.8 is up to date.
update product_config done
Cloning into '/NVMPs/c225v2/NVMP/sdk/wlan/icommsemi'...
fatal: Remote branch nvmp_release_1.8 not found in upstream origin
root@f2aca03832ff:/NVMPs/c225v2/NVMP#
root@f2aca03832ff:/NVMPs/c225v2/NVMP# ./scripts/build_main.sh -p c225v2 -s all -n
product: c225v2
product_config: updating product_config source ...

Current branch nvmp_release_1.8 is up to date.
Already on 'nvmp_release_1.8'
Your branch is up-to-date with 'origin/nvmp_release_1.8'.
Current branch nvmp_release_1.8 is up to date.
update product_config done
Cloning into '/NVMPs/c225v2/NVMP/sdk/wlan/icommsemi'...
fatal: Remote branch nvmp_release_1.8 not found in upstream origin
root@f2aca03832ff:/NVMPs/c225v2/NVMP#
root@f2aca03832ff:/NVMPs/c225v2/NVMP# ./scripts/build_main.sh -p c225v2 -s all -n
product: c225v2
product_config: updating product_config source ...
```
3. 如何编译固件？
4.
5. 这里可以确认编译还有没有运行。
6. 

7. 因为别人在代码里下毒导致的编译失败。不要看下面的“failed”，直接看 error

8. 

9. 编译docker和编译固件的历史踩坑：
10.

```
git clone "ssh://liuyuanhao@gerrit-ce.tp-link.com:29418/IPC/NVMP" && scp -p -P
29418 liuyuanhao@gerrit-ce.tp-link.com:hooks/commit-msg "NVMP/.git/hooks/"

# 切换到电池机分支 bc_develop_220927
# 先remove掉之前的docker
./scripts/build_main.sh -p c460v1 -s all -n

./nvmp/docker/build_docker.sh c460v1

tplink@tplink-vm:~/audio_demos/tmp/NVMP$ ./scripts/build_main.sh -p c460v1 -s
all -n
product: c460v1
Cloning into '/home/tplink/audio_demos/tmp/NVMP/nvmp/product_config'...

remote: Counting objects: 94, done
remote: Finding sources: 100% (94/94)
remote: Total 90181 (delta 11), reused 90129 (delta 11)
Receiving objects: 100% (90181/90181), 422.20 MiB | 7.58 MiB/s, done.
Resolving deltas: 100% (45551/45551), done.
Updating files: 100% (2886/2886), done.
commit-msg
100% 2174    13.5KB/s   00:00
clone product_config done
Cloning into '/home/tplink/audio_demos/tmp/NVMP/sdk/wlan/RTL'...
remote: Total 9277 (delta 0), reused 9277 (delta 0)
Receiving objects: 100% (9277/9277), 34.06 MiB | 7.25 MiB/s, done.
Resolving deltas: 100% (4265/4265), done.
commit-msg
100% 2174    13.2KB/s   00:00
clone wlan module done
Cloning into '/home/tplink/audio_demos/tmp/NVMP/sdk/soc/ssc30x'...
```

```
remote: Counting objects: 6, done
remote: Finding sources: 100% (6/6)
remote: Total 139099 (delta 0), reused 139096 (delta 0)
Receiving objects: 100% (139099/139099), 2.11 GiB | 6.68 MiB/s, done.
Resolving deltas: 100% (28390/28390), done.
Updating files: 100% (115201/115201), done.
commit-msg
100% 2174    13.4KB/s   00:00
clone sdk done
ams: Now cloning ams source ...
Cloning into '/home/tplink/audio_demos/tmp/NVMP/nvmp/ams_package'...
remote: Counting objects: 5, done
remote: Finding sources: 100% (116046/116046)
remote: Total 116046 (delta 61741), reused 115950 (delta 61741)
Receiving objects: 100% (116046/116046), 2.81 GiB | 5.83 MiB/s, done.
Resolving deltas: 100% (61741/61741), done.
Updating files: 100% (8773/8773), done.
commit-msg
100% 2174     9.2KB/s   00:00
avts: Now cloning avts source ...
Cloning into '/home/tplink/audio_demos/tmp/NVMP/nvmp/tp_package/avts'...
remote: Counting objects: 21, done
remote: Finding sources: 100% (21/21)
remote: Total 32119 (delta 0), reused 32102 (delta 0)
Receiving objects: 100% (32119/32119), 136.11 MiB | 5.00 MiB/s, done.
Resolving deltas: 100% (18181/18181), done.
commit-msg
100% 2174    11.3KB/s   00:00
cap: Now cloning cap source ...
Cloning into '/home/tplink/audio_demos/tmp/NVMP/nvmp/tp_package/cap'...
remote: Total 23575 (delta 0), reused 23575 (delta 0)
Receiving objects: 100% (23575/23575), 64.58 MiB | 6.22 MiB/s, done.
Resolving deltas: 100% (11324/11324), done.
commit-msg
100% 2174    11.9KB/s   00:00
nsd: Now cloning nsd source ...
Cloning into '/home/tplink/audio_demos/tmp/NVMP/nvmp/tp_package/nsd'...
remote: Counting objects: 127, done
remote: Finding sources: 100% (127/127)
remote: Total 22940 (delta 8), reused 22839 (delta 8)
Receiving objects: 100% (22940/22940), 21.60 MiB | 6.08 MiB/s, done.
Resolving deltas: 100% (12699/12699), done.
commit-msg

# 如果看到 Current branch bc_develop_220927 is up to date.
update product_config done
unspported wlan type
说明 wlan 和其他模块没有拉全！后面编译 docker 会报错


./nvmp/docker/build_docker.sh c460v1
# 屏幕截图 2025-08-19 094007.png
```

在创建docker时遇到报错 failed to compile:255 / 127 / ...
很可能是之前几步的仓库没有拉完整

```
./nvmp/docker/build_docker.sh c460v1
Products: c460v1
Builder: nvmp_builder_16.04
now building the container  ...
Build image ...
Build image for product c460v1 ...
rm: cannot remove '/nvmp_source_dir/nvmp/tmp/info/.files-packageinfo-569555':
Permission denied
make[2]: *** No rule to make target '/nvmp_source_dir/nvmp/tmp/info/.files-
packageinfo-588082', needed by '/nvmp_source_dir/nvmp/tmp/info/.files-
packageinfo.stamp'.  Stop.
/nvmp_source_dir/nvmp/include/toplevel.mk:69: recipe for target 'prepare-
tmpinfo' failed
make[1]: *** [prepare-tmpinfo] Error 2
make: *** [world] Error 2
/nvmp_source_dir/nvmp/include/toplevel.mk:210: recipe for target 'world' failed
failed to compile: 255
```

对于 “rm: cannot remove '/nvmp_source_dir/nvmp/tmp/info/.files-packageinfo-569555': Permission denied”, 是因为上次的编译中间结果未清理，手动进入'/nvmp_source_dir/nvmp/ 执行 make clean 命令即可。 再执行 ./nvmp/docker/build_docker.sh c460v1

```
#报错Error response from daemon: Get "https://rdsource.tp-link.com:8088/v2/":
http: server gave HTTP response to HTTPS client
Error response from daemon: Get "https://rdsource.tp-link.com:8088/v2/": http:
server gave HTTP response to HTTPS client ?
```

尝试关闭免密登录， mv ~/.ssh/authorized_keys ~/.ssh/authorized_keys.bak 。没有效果。

尝试配置docker源 https://rdsource.tp-link.com/docker_help.html, 需要配置文件和登录公司 docker 仓库
显示“Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store

Login Succeeded” 登录成功！

```
#没有创建 nvmp_source_dir 文件夹却在跑 build_docker 脚本？
make[1]: *** [/nvmp_source_dir/nvmp/staging_dir/target-arm-openwrt-linux-
uclibcgnueabihf-c460v1/stamp/.target_compile] Error 2
target/Makefile:16: recipe for target '/nvmp_source_dir/nvmp/staging_dir/
target-arm-openwrt-linux-uclibcgnueabihf-c460v1/stamp/.target_compile' failed
make[1]: Leaving directory '/nvmp_source_dir/nvmp'
/nvmp_source_dir/nvmp/include/toplevel.mk:210: recipe for target 'world' failed
make: *** [world] Error 2
failed to compile: 255
```

答：因为 nvmp_source_dir 文件夹 在docker容器里面啊小傻瓜。跑到这个报错说明 docker 已经创建成功了。

！！# 最后 failed 的解决方案是，弃用 build_docker 的make，【使用 docker 的 attach shell】改用docker内部的 make 命令
推荐使用

```
#make PR_NAME=c460v1 package/ams/compile V=s -j10
touch /nvmp_source_dir/nvmp/staging_dir/target-arm-openwrt-linux-uclibcgnueabihf-c460v1/stamp/.libjpeg_installed
make[2]: Leaving directory '/nvmp_source_dir/nvmp/tp_package/lib/libjpeg'
make[1]: Leaving directory '/nvmp_source_dir/nvmp'
/nvmp_source_dir/nvmp/include/toplevel.mk:210: recipe for target 'package/ams/compile' failed
make: *** [package/ams/compile] Error 2?
```

检查各 sdk 的节点是否正确
访问 \\exfile.tp-link.com\联洲国际研发\public\外销Camera提测\TapoCam\提测-勿删 的对应机型，查看最新一次编译的 “20250815.compile_commit_id”, 形如

```
[product_config]nvmp_bc_release:55a5980
[nvmp]nvmp_bc_release_1.8.1:cc394a8
[wlan]bc_develop_220927:858ad30
[sdk]nvmp_bc_release_1.8.1_aov:f9a191f
[ams]master:92a6e33
[avts]nvmp_bc_release_1.8.1:998d2be
[cap]nvmp_bc_release_1.8.1:96ce4bc
[nsd]nvmp_bc_release_1.8.1:d3d02f0
```

其中 左侧是该模块使用分支，右侧是此处的 commit，可以cd进对应的目录，用 git log 查询当前 commit，用 git reset --hard commit-id 尝试切换到对应的 commit

带机型做 make。

```
#make clean
#make PR_NAME=c460v1 V=s -j4
```

其中 c460v1 是机型； V=s 是详细输出； -j4 是多线程编译

```
make[2]: *** [target/linux/compile] Error 2
make[2]: Leaving directory '/nvmp_source_dir/nvmp'
target/Makefile:16: recipe for target '/nvmp_source_dir/nvmp/staging_dir/target-arm-openwrt-linux-uclibcgnueabihf-c460v1/stamp/.target_compile' failed
make[1]: *** [/nvmp_source_dir/nvmp/staging_dir/target-arm-openwrt-linux-uclibcgnueabihf-c460v1/stamp/.target_compile] Error 2
make[1]: Leaving directory '/nvmp_source_dir/nvmp'
/nvmp_source_dir/nvmp/include/toplevel.mk:210: recipe for target 'world' failed
make: *** [world] Error 2?
```

答：需要往前翻报错！！细节报错才有用，现在这个failed 没有意义

```
# make 过程遇到需要选 y/n


# 报错 IOError: [Errno 2] No such file or directory: './sc/system/sys/pub/sys_sys_version.h'
tng/product.mak:58: recipe for target 'pre_process' failed
```

```
# 怀疑是 sdk 的某个分支没有支持，改用 c220v1 的固件编译。
# 更新 node.txt 到 c220 版本
# python3 节点切换.py ~/audio_demos/NVMP /home/tplink/audio_demos/NVMP/Gerrit/
node.txt
From ssh://gerrit-ce.tp-link.com:29418/IPC/NVMP/MSTAR/ssc30x
 * branch                develop_sz0519 -> FETCH_HEAD
Already up to date.
fatal: ambiguous argument 'b5f6eec': unknown revision or path not in the
working tree.
Use '--' to separate paths from revisions, like this:
'git <command> [<revision>...] -- [<file>...]'
sdk b5f6eec node wrong!!!
节点报错 sdk b5f6eec node wrong, 可能是该commit节点是临时节点

# 切换到分支 bc_develop_220927 【后来发现这 tm 是电池机分支】
#./scripts/build_main.sh -p c220v2 -s all -n
Current branch bc_develop_220927 is up to date.
update product_config done
unspported wlan type
报错 wlan 模块不可用

## 换条路线，重新下载仓库。防止脏东西
ref: [NVMP 代码拉取与编译（2025)] https://pdconfluence.tp-link.com/pages/
viewpage.action?pageId=71101728

按机型创建仓库
# mkdir ~/audio_demos/NVMPs/c220v2
# git clone "ssh://liuyuanhao@gerrit-ce.tp-link.com:29418/IPC/NVMP" && scp -p
-P 29418 liuyuanhao@gerrit-ce.tp-link.com:hooks/commit-msg "NVMP/.git/hooks/"
Receiving objects: 100% (91736/91736), 870.96 MiB | 4.80 MiB/s, done.
Resolving deltas: 100% (43965/43965), done.
Updating files: 100% (24192/24192), done.
commit-msg
100% 2174     6.1KB/s   00:00

切换到远程常电机分支  develop_sz0519（一般情况下，若电池机则bc_develop_220927分支，常电
机则develop_sz0519分支）
# git checkout -b develop_sz0519 origin/develop_sz0519

拉取所有代码，机型 -p c220v2 。 -n 意思是不编译
# ./scripts/build_main.sh -p c220v2 -s all -n
# ./scripts/build_main.sh -p c225v2 -s all -n
nsd: Now cloning nsd source ...
Cloning into '/home/tplink/audio_demos/NVMPs/c220v2/NVMP/nvmp/tp_package/nsd'..
.
remote: Counting objects: 58, done
remote: Finding sources: 100% (58/58)
remote: Total 23434 (delta 2), reused 23394 (delta 2)
Receiving objects: 100% (23434/23434), 21.97 MiB | 7.81 MiB/s, done.
Resolving deltas: 100% (12996/12996), done.
commit-msg
100% 2174    13.6KB/s   00:00
```

```
Cloning into '/home/tplink/audio_demos/NVMPs/c220v2/NVMP/nvmp/tp_package/cap/
src/modules/fac_test'...
fatal: Remote branch develop_sz0519 not found in upstream origin
【暂时无视】

# ./nvmp/docker/build_docker.sh c220v2
tplink@tplink-vm:~/audio_demos/NVMPs/c220v2/NVMP$ ./nvmp/docker/build_docker.sh
c220v2
Products: c220v2
Builder: nvmp_builder_16.04
ccache dir is: /home/tplink/.ccache
8488579ae4e36002c32ac8be8b78ba62300ceea3a673914c6712b90768232765
now building the container  ...
# YES/NO 选项【暂时回车选择默认】
GKI Dummy config options (GKI_HACKS_TO_FIX) [N/y/?] n
RTL_TSS_SUPPORT (KERNEL_TSS_SUPPORT) [Y/n/?] y
Special fix for tapo wq wlan bridge (TAPO_WQ_CONNECT_NET) [N/y/?] (NEW)
报错？
/nvmp_source_dir/nvmp/include/toplevel.mk:277: recipe for target 'world' failed
failed to compile: 255

补充环境，在docker内和docker外，逐行安装（需要处理 yes/no）
# sudo apt-get update
sudo apt-get install bison
sudo apt-get install kmod
sudo apt-get install qt5-default
sudo apt-get install ccache
sudo apt-get install libgtest-dev
sudo apt-get install libopencv-dev
sudo apt-get install libmbedtls-dev
sudo apt-get install libunwind-dev

# sudo apt autoremove
移除  gir1.2-goa-1.0:amd64  看起来是冲突的库

手动进入 docker ，重新运行
# cd /nvmp_source_dir/nvmp
# make clean
不知道为什么，但好像 c220 需要用 -j1 编译？有一个相关的 warning
这个命令时间很长。
# make PR_NAME=c220v2 V=s -j1
得到 .bin 产物



小结：
1. 发现并不需要手动在不同 sdk 拉不同的库，甚至不需要用 .py，反而使用 build_main 得到的
 product_config 和 sdk 就是可编译的。之前实际上是卡在了 外部依赖库 和不完备的
 build_docker.sh。
2. build_docker.sh 最后失败了也没关系，只要容器出来了就行。
3. 常电机和电池机分支是个大的坑点！
```

```
## 编译分支正确的固件bin
切换分支。 最好在 Jenkins2 拿最新的，提测的 commit 不一定有效。或者只改分支，然后 pull 到
最新的
# python3 分支节点切换.py /home/tplink/audio_demos/NVMPs/c220v2/NVMP /home/
tplink/audio_demos/NVMPs/c220v2/compile_commit_id.txt

 * branch              nvmp_release_1.8.1 -> FETCH_HEAD
Already on 'nvmp_release_1.8.1'
Your branch is up to date with 'origin/nvmp_release_1.8.1'.
From ssh://gerrit-ce.tp-link.com:29418/IPC/NVMP/NSD
 * branch              nvmp_release_1.8.1 -> FETCH_HEAD
Already up to date.




##编译音频 pc_dbug
cd ams/pc_dbug/aqi:
python3 compile.py aec3 ns3 agc tpaec3ns3
  File "compile.py", line 18
SyntaxError: Non-ASCII character '\xe6' in file compile.py on line 18, but no
encoding declared; see http://python.org/dev/peps/pep-0263/ for details
这是因为 python2 默认支持 ASCII 编码，实际应当使用 UTF-8 编码。 所以改用 python3

编译 pc_dbug 代码
./compile.sh -t --dla
CONFIG_MACRO_DLA= -DDL_IMAGE_TYPE_YUV420SPNV21 -DMODEL_DESC -DDL_DRAW_BBOX
-DDLA_PKGD_BY_OD -DFLM_P98 -DDLA_FR -DDLA_FSS -DDLA_SNAPSHOT -DDLA_PSS
-DDLA_FCD -DJPEG_SUPPORT -DCAMEOS_SUPPORT -DDUMP_RAWDATA_SUPPORT
CONFIG_MACRO_QT= NO_GUI DL_DRAW_BBOX DL_IMAGE_TYPE_YUV420SPNV21
./compile.sh: line 133: qmake: command not found
make: *** No targets specified and no makefile found.  Stop.

手动安装 qmake :   sudo apt-get install qt5-default  # Qt5 默认版本
再次运行./compile.sh -t --dla
make: ccache: Command not found
Makefile:992: recipe for target 'tests.o' failed
手动安装  sudo apt-get install ccache
#gtest
  sudo apt-get install libgtest-dev
#opencv2
  sudo apt-get install libopencv-dev
sudo ln -s /usr/include/opencv4/opencv2 /usr/include/
使用链接失败。
直接mv出来
sudo mv /usr/include/opencv4/opencv2 /usr/include/opencv2
sudo mv /usr/include/opencv2 /usr/include/opencv4/opencv2
也不行。
 链接到文件夹
sudo ln -s /usr/include/opencv4/opencv2 /usr/include/opencv2
sudo ln -sf /home/tplink/libtorch /opt/libtorch
```

找不到 <torch/cuda.h><torch/script.h>???
很奇怪，改用 ./compile.sh -d 或者 ./compile.sh -t 就可以直接编译出结果？？？
好像是因为 --dla 会把dla的配置拉过来，dla的buildroot.config 比较复杂，涉及深度学习

```
mv /home/tplink/libtorch /usr/local/lib/libtorch
sudo ln -s /usr/local/lib/libtorch/include/torch  /usr/local/bin/torch
```

```
./compile.sh -d -p c460v1 事实上 ams/config文件夹内只有默认那个机型 所以这里不应当指定-
p
ion/vdr_flm_dla -I../../../src/adr/utils -isystem /usr/include/x86_64-linux-
gnu/qt5/QtWidgets -isystem /usr/include/x86_64-linux-gnu/qt5/QtGui -isystem /
usr/include/x86_64-linux-gnu/qt5/QtCore -I. -I. -I/usr/lib/x86_64-linux-gnu/
qt5/mkspecs/linux-g++ -o topaint.o topaint.cpp
Could not create output file
使用 sudo 取得权限

算法ams_package 不需要进docker，推荐在主机上编译

#4    Object "/usr/lib/x86_64-linux-gnu/libQt5Gui.so.5.12.8", at 0x7fae7ba2e707,
 in QGuiApplicationPrivate::createEventDispatcher()
#3    Object "/usr/lib/x86_64-linux-gnu/libQt5Gui.so.5.12.8", at 0x7fae7ba2d7ad,
 in QGuiApplicationPrivate::createPlatformIntegration()
#2    Object "/usr/lib/x86_64-linux-gnu/libQt5Core.so.5.12.8", at
0x7fae7b449aac, in QMessageLogger::fatal(char const*, ...) const
#1    Source "/build/glibc-B3wQXB/glibc-2.31/stdlib/abort.c", line 79, in abort
[0x7fae7ae7c858]
#0    Source "../sysdeps/unix/sysv/linux/raise.c", line 51, in raise [0x7fae7ae
9d00b]
Aborted (Signal sent by tkill() 13916 1000)
Aborted
```

```
cp ./linux/config/c310v3/buildroot_basic.config ./linux/config/c310v3/
buildroot.config
```

```
重新 compile --dla 的时候失败，使用
sudo ln -sf /home/tplink/libtorch /opt/libtorch
https://pdconfluence.tp-link.com/pages/viewpage.action?pageId=71102329
```

```
/usr/bin/ld: build_dir/linux/libraries/libdla/libtorch_utils.o: in function
`c10::ArrayRef<c10::SymInt>::debugCheckNullptrInvariant()':
/opt/libtorch/include/c10/util/ArrayRef.h:57: undefined reference to
`c10::detail::torchInternalAssertFail(char const*, char const*, unsigned int,
char const*, char const*)'
```

```
collect2: error: ld returned 1 exit status
make: *** [Makefile:316: main] Error 1
build failed: 2
```

答：把 build_dir 和 build_qt 删掉后，再重新编译。因为 原本的CMake结果是脏的。

确认export LD_LIBRARY_PATH=/home/tplink/libtorch/lib:$LD_LIBRARY_PATH 正确执行
检查 echo ${LD_LIBRARY_PATH}

7 项目笔记

7.1 25年08月27日 合作大作业

7.1.1 总结

1. 粗略翻了10组汇报中感兴趣的部分，第 4 组的客户端页面是用 pyQT5+qtfluentwidgets写的 ，效果非常好。
2. 算法层面，我是唯一用 基于OCR结果识别工牌 的方法的人，毕竟这个方法在网上都搜不到；其实 FaceInsight + PaddleOCR 的方法，除去初始化的运行效率是和我一样快的（300~400ms），因为 ta 人脸检测只需要 20ms 不再是瓶颈了。而且 ta 还是用 python 写的算法，我是改 C++ 才优化到 300ms ；主流算法实现就是 YOLO、基于各种算子的边缘检测，然后 OCR 随便调个库也没调研的那种。
3. 我回想起几个月前做 extraceEntity 的时候，被批“没有把算法做到极致”。可能那时候我把重心放在了前端页面和作为产品的优化吧。

7.1.2 题目要求

- （必须）我司摄像头通过网络连接到PC，PC 获取并显示摄像头画面
- （必须）PC 端程序实时识别我司工牌，包括提取头像、工号、姓名
- （可选）识别成功后记录打卡信息，并控制摄像头发出声音提示打卡成功
- （可选）可按打卡人身份查询、统计近期的打卡记录

链接: 算法实践大作业&分组安排[7]

7.1.3 实现思路

1. 核心模块：【考虑c语言实现】使用 FFmpeg 实现视频拉流，使用 OpenCV 定位工牌（画检测框），使用 tesseract 实现文字OCR（可能需搭配训练数据）。
    a. 对比：c语言效率更快，但更难； python 实现起来很快，应该秒了 。
2. 风险点：tesseract 和 OpenCV 可能需要从外网安装。
3. 评判标准：实时性（延迟时长，预期为毫秒级），误报率&召回率（预期大于 97%）。
4. 评分标准：边界情况、用户界面、识别速度、识别准确率
5. 使用名单保存公司名单+工号。但是需要加随机名字进去混淆，防止特调的嫌疑。
    a. 明确记录三个常量，保留8位精度

7 https://pdconfluence.tp-link.com/pages/viewpage.action?pageId=166829667

6. theta = np.radians(70.2126667817932)

   frac_bc_ab = 3.3637369367866397

   kw = 2.532680442084436# 比例尺

   2. 取点　auto p3=boxes[anchor_index].second[3];

   auto p2=boxes[anchor_index].second[2]; 作为三角形的点A 和点B。其中角ABC=theta

   3. 计算三角形的点C作为 face_lefttop。计算w=kw*len_AB

   4. return w,face_lefttop

7.1.4 思路【中期】

1. 压缩到 360p 再搞
2. 姓名数据库+工号数据库
3. 需要改成socket通信
4. 【因为 extract face 依赖 文本框位置，所以并行不了】并行多线程实现 提取工牌信息extract_info和 图像矫正+选取头像位置 extract_face
5. 长宽比过滤（启发式） 约 2.5
6. 仿射变换 图像摆正
7. 关键帧（软件层）
8. 可能考虑用 yolo v8 实现？ 调研发现用cpu算力，一定远慢于10秒
9. 公司用不了 torch？
10. 关于目标检测
    a. 高斯降噪
    b. HSV青色提取
    c. 形态学处理
    d. 阈值分割
    e. 边缘检测 - opencv findContours
    f. 轮廓框过滤
        i. 启发式过滤
        ii. 仿射变换
11. 关于OCR
    a. 感知哈希
    b. Tesseract OCR
    c. SVM字符识别
    d. 采用opencv**拉普拉斯变换后计算图片方差**的方式衡量图片清晰程度，一般方差越大图像越清晰
    e. 躺好点，让你 **RapidOCR**大爹发挥，0.9秒一张无需目标检测的4k图+超高准确率

12. 姓名和工号信息提取
    a. 使用 正则式匹配，nameList
    b. 姓名中的错别字修正？
13. 上下文匹配 整合一段时间内的信息进行匹配，可以提高识别效率
14.

    多卡同时识别 / 排除多卡误打卡
15. 头像定位: 基于确定的姓名框的位置，而仅仅基于一个位置确定头像，会出现偏移的问题。因此考虑到定位不准确的问题，后续可以增加多个特征进行定位，包括姓名框，logo等。【求平均】

16. 透射变换和仿射变换，能对360度各方向旋转的工牌进行准确的打卡。由于使用的是仿射变换，因此对于前后倾斜的工牌会随着倾斜角度的增大造成打卡能力的下降。进一步可以在对倾斜图像的处理上进行研究，增加透射变换，提升识别准确率。 【注意：如果做长度的变换，文字会变形。 因为无法知道预期结果的文字矩形】

17. 并发：对于 multiprocessing 和 concurrent.futures 都能处理CPU密集型任务的并发，这里选用更新的 concurrent.futures。 而 asyncio 主要用于 网络高并发任务。

7.1.5 调研笔记

1. OCR
    a. 以下所说数据集是指包含摄像头拍摄和手机拍摄的若干张不同角度工牌（-135°~+90°）、不同距离（20~60cm）图像组成的数据集。
    b. TesseractOCR 尝试在 --psm 和 --oem 两方面调参，都不达预期，并且需要自己做预处理灰度化等等操作后，才能稍微提高效果到可用。出图挺快的，最快0.5秒。
    c. EasyOCR 跑的很慢，在 cpu 上十秒才能出结果。没有在数据集上测试。pass
    d. Rapid OCR 出图相当快，0.9秒一张，并且在数据集上成功率很高。
    e. 根据广州软件院发布的 OCR开箱测评报告 的自然场景部分，得到的结论和上述实践相匹配。

7.2 25年08月18日 AMS作业：编译一个AMS模块

1.
```
tests.h:9:10: fatal error: gtest/gtest.h: No such file or directory
    9 | #include "gtest/gtest.h"
      |          ^~~~~~~~~~~~~~~
compilation terminated.
make: *** [Makefile:1037: tests.o] Error 1
make: *** Waiting for unfinished jobs....
In file included from backward.cpp:39:
backward.hpp:411:10: fatal error: libunwind.h: No such file or directory
  411 | #include <libunwind.h>
      |          ^~~~~~~~~~~~~
compilation terminated.
```
2. 这里的 gtest.h 是 Google Test ， libunwind.h 也是外部依赖，不是公司内的依赖。自己用命令行安装即可。 或者使用 docker 里面有这个依赖。 docker的本质是进程隔离。
3.
```
# 安装 gtest.h
sudo apt-get install libgtest-dev  # 安装头文件和源码
cd /usr/src/gtest
sudo cmake CMakeLists.txt  # 编译源码
sudo make
sudo cp lib/*.a /usr/lib  # 将静态库复制到系统库目录

# 安装 libunwind.h
sudo apt-get install libunwind-dev # 安装主库
```

7.3 25年08月13日 竞品对比

1. 新人任务的竞品对比，应当更多用户体验，而不是局限于所在专业。
2. 人声由基频及其倍数谐波组成。如果谐波因为降噪等原因损失，就会发生 谐波失真， 听起来有失真感。
3. Tapo C250 vs EZVIZ C6c 4K
4. EZVIZ 需要开热点后，才能连接。
    a. 吐槽：遮蔽模式→ 解除遮蔽模式 无法打断
    b. 不支持在主页预览，主页看到的画面一分钟刷新一次
5. Tapo 可以**局域网连接**。 蓝牙自动连接失败。
    a. Tapo配对踩坑
        i. Tapo_CAM_XXXX 的WIFI不会立刻刷出来，等待5~15秒是正常的。
        ii. 并且需要在配置完设备wifi后，将手机wifi从 TAPO_CAM_XXXX 切换到 所连接的 2.4GHz Wi-Fi

iii. **并且关闭流量！！**

b. 电机基本没有声音
c. 可能是受局域网环境影响，声音卡顿感明显。
d. 自动光线识别不可用。 经验证 EZVIZ 的自动切换白天模式是可用的。

8 练手任务 *2025.7.28-2025.8.4*

请使用**C语言**实现一个低通滤波器，并使用该滤波器将一个**16k采样率**的音频文件的**6750-8000频段**的信号**滤除**。
需要描述关键步骤的原理。
可能有不止一种实现办法【不同的分帧加窗 or 不同的对高于截止频率的处理】，在有余力的情况下尽可能实现多的方案。

可能需要思考：
1. 低通滤波器的原理 【FFT之后将高频按某种运算缩减，即“高于截止频率”的部分 置为0、衰减或平滑 ；以及非fft的滤波器方法】

【低通滤波在时域上的表现？】

—— 滤波器考虑 FIR 有限脉冲 or IIR 无限脉冲
2. 如何使用C语言编译出一个可执行文件
3. 如何读取音频文件 如何处理音频信号 如何写入音频文件 【（无损文件）wav文件的读入，处理，写入】
4. 如何呈现滤波效果 【结果频谱可视化，预期高频=0 or 高频衰减】

可能用得到的关键字：分帧 加窗 FFT IIR FIR 卷积 采样率

需要提交：
代码 文档（记录实现的方案与效果） 可执行文件

时间：
一周

9 竞品测试笔记

1. **注意不能用流量访问 Tapo App。关闭流量后，连接设备所在局域网的 Wifi，才会显示设备。**
2. dms 可以查询项目报告 https://dms.tp-link.com/pages/project/Project.jsp?MiljJGt4EGJpe3giJwIpKCoEOjw1bH0PICI5LGt5YiM4KDo+J3V/DRwOGBUNGgsPcWJ6ICpuZxR+fHl4YQ=
3. 上周测试竞品 萤石S10踩了个坑：

   默认情况，app录制正常可用；音频设置中打开人声增强，app录制的视频会没有声音，手机自带录屏可用；后续测萤石音频效果前注意一下

9.1 25年08月 新人工作日报留档

刘元昊	周一	7.21	1.参加公司级培训：公司简介、公司制度规范 2.参加公司级培训：新手指南、工作规范、E-mail使用规范	1.了解公司基本信息、规章制度以及企业文化 2.了解工作规范、E-mail使用规范，完成从学生思维到职场思维的基本转换
	周二	7.22	1.参加公司级培训：廉洁自律、团队合作 2.参加公司级培训：有效沟通 3.初次抵达工位，完成电脑配置 4.参加公司级培训：PPT应用技巧	1.了解公司廉洁自律的要求与奖惩措施 2.了解团队合作的基本技巧与公司的当责文化，了解有效沟通的基本技巧 3.参考《2025年新入职同事电脑设置指引》配置电脑 4.了解PPT的相关制作技巧
	周三	7.23	1.参加公司级培训：计算机信息安全及网络管理规范、职场学习 2.参加公司级培训：公司产品介绍	1.了解公司计算机信息安全与网络管理规范的相关制度，深刻认识到网络安全的重要性 2.了解职场学习的重要性以及职场学习的相关方法 3.了解公司各部门研发的相关产品

周四	7.24	1.参加公司级培训：情绪与压力管理、思维导图使用技巧 2.参加公司级培训：个人数据合规保护 3. 整理课表到日历，并预习后续一周课程内容。 4. 配置虚拟机环境，**配置开发环境。**	1.了解情绪与压力管理的基本方法，认识到不仅需要管理负面情绪，也要对其他情绪进行管理 2.学习思维导图使用技巧，使用XMind制作思维导图，体会到思维导图对于发散性思维的促进作用 3.了解个人数据合规保护的相关制度
周五	7.25	1. 参加公司级培训：专利基础培训、ISO9000培训 2. 根据往年资料，**梳理音频处理流程和音频算法的思维导图**，详见 Algo 笔记本 (see page 3)	1.学习并了解实验室的相关制度规范与网络配置技巧 2.了解专利相关知识与公司对申请专利的奖励制度，了解ISO9000规范 3. 对3A算法和知识的总框架有了明确的理解。
周一	7.28	1. 参加公司线下培训：实验室制度+**实验室安全知识** 2. 参加公司线下考核：实验室制度+**实验室安全知识考核**	1. 了解了实验室相关内容。期待去实验室。
周二	7.29	1. 参加公司线下培训：开发工具软件使用指南 2. 参加公司线下培训：版本库管理与持续集成管理 3. 参加公司线下考核：实验室制度+**实验室安全知识考核（补考）**	1. 了解了公司市场使用的开发工具，包括串口软件，文件传输等等 2. 了解了如何加载、处理和保存音频数据 3. 了解了如何管理代码库以及版本，学会了一些git指令
周三	7.30	1. 参加公司线下培训：编码规范-C 2. 参加公司线下培训：C语言陷阱 3. 尝试完成新人任务代码，进度 70%。	1. 了解了公司编码的规范，对 c 有了更多的了解 2. 因为自己直接对 C++ 比较熟悉。从直接控制硬件的程度说： C > C++ > Python。从原先的 freopen 和无需关注内存，到 c语言内自行通过 fread+malloc+free控制文件操作和内存，需要更精细化一些。但思路变化不大。

周四	7.31	1. 继续完成新人任务代码，代码结构完成100%，算法实现完成 2/3 。 2. **找到之前标程的问题**。因为我的测试信号就两个信号，然后截止频率设的6750 ，FIR系数又是用窗函数截断做的。 导致的结果就是，因为7000Hz只比截止频率大一点，所以还没来得及衰减。 测试信号只有两个正弦波，也没一眼看出来是截止频率参数的问题。 3. **白噪 = 无限多个不同频率的正弦波+随机相位偏移**得到？ 思考并调研了一下这个问题，即无限多个正弦波，能否通过控制频率均匀随机、相位均匀随机，得到一个结果恰好为 0 的时域音频。 4. 发现另一个错误，因为我一开始没理解那个矩阵窗方法的 FIR 系数公式，导致实际在频域点乘了个完整的窗函数进去，而不是半个窗函数。	1. 中途使用 32位/64位浮点数运算，防止数据溢出，减少精度丢失。采样深度 16位 对应 int16_t，即标准的16位**有符号**整型数据类型。 1. 音频信号是交流信号，包含正负幅值（如声波的压缩与稀疏部分） 2. 小心高频信号，如果高于一半采样频率，就会发生信号混叠。 3. 频域点乘等价于时域卷积； 时域点乘等价于频域卷积；这两个结论可以很好的推出 FIR 的系数构造。 4. 本质上 FIR 是一个简单线性系数构造； IIR 是一个形如 \sigma {x_i * a_i} == \sigma {y_i * b_i} 的式子，其中x是输入，y是输出，处理过程类似 DP。所以 IIR 也可以转化成矩阵乘法表示。
周五	8.01	1. 完成音频算法的 Week1 新人任务和 《C 语言规范》课程的本周作业，实现了基于分块的 memcpy 优化。 2. 导出Week1 新人任务的实验结果，编写文档 readme.md[8]。	1. 处理内存重叠和分块+remaining的问题。 本质其实是空间换时间。最后发现没有-O3编译优化的话，几乎不可能超过c语言原生的 memcpy。

8 http://readme.md

周一	8.4	1. 上午上课 交叉编译和 makefile 2. 周会16~18点，记录了很多专业知识相关笔记，详见周二-“**学习音频`概念**” 3. 阅读	1. .wav解码，如果没有考虑立体声的话会出问题。 2. c 项目总结 a. **内存管理原则**：仅对 malloc/calloc/realloc 分配的内存调用 free，其他变量由系统自动处理。 b. /usr/bin/ld: /tmp/cczdgixI.o: in function `min': audio_io.c:(.text+0x0): multiple definition of `min'; /tmp/ccdnV2eK.o:generate.c:(.text+0x0): first defined here? i. 这表示 min 被重复定义了，是数学库里自带了 fmin 导致的。 一般 max\min 应当用 #define 定义。 ii. “-lm” 要写在最后，链接数学库。
周二	8.5	1. 上午上课。 嵌入式linux 基础 2. 阅读 IIR 的数字滤波器设计的推导过程。 3. 顺着 【音频】新人培训[9] 补全音频专业知识。 4. “嵌入式编程”的作业习题 T1	1. IIR 的数字滤波器设计 ，原理牵扯了很多数学分析的内容，包括魏尔斯特拉斯定理 2. **学习音频`概念** a. 总谐波失真（THD） b. 音频麦克风的线性性（Linearity） c. DSP硬件 （数字信号处理器） d. 裸流丢帧 e. 音量映射/响度映射
周三	8.6	1. 嵌入式linux基础作业习题 T1收尾，完成T2+T3 2. T3要求编写 “注册字符设备 my_device，支持带 cap_lock 的读写操作” 的内核模块	1. Shell 没有想象中灵活，对运算的支持只能通过 expr 或者内嵌 c 的代码。 2. Linux kernel 模块的编写比预期简单。总体非常开放。
周四	8.7	1. 完成嵌入式linux基础作业习题T3~T5，写实验报告文档，并导出为 pdf。 2. 阅读**《实时语音处理实践指南》**并做笔记。 3. 调研低采样率引入的**混叠噪音**	1. 相比于视频的码率和分辨率不断攀升，音频算法因为采样定理的存在，保证了 48k 的**采样率**就足以无失真的保存频段在 0~20kHz 的模拟信号。对于音频响度，16bit~64bit的**采样深度**也足以提供远大于人耳极限的动态范围。 2. 这带来一个优点，即音频技术的稳定性。 3. 这并不是说音频技术是止步不前的，深度学习等可用于音频算法的新技术在这几年涌现；而是说音频问题是一个边界更明确的问题。

9 https://pdconfluence.tp-link.com/pages/viewpage.action?pageId=157615282

周五	8.8	1. 上课《IPC产品分析》+《音视频算法介绍》。 2. 跟随同事在实验室旁观测试。 3. 手写证明折叠频率公式。	1. 对音频处理领域有了框架上的了解。
周一	8.11	1. 培训：NVMP基础及关键模块介绍 2. 培训：AMS 算法架构 3. 测试萤石 EZVIZ C6c 4K 样机，探索 EZVIZ app 的全部功能。	1. NVMP 是我第一个接触的单进程框架（单体架构）。从程序运行逻辑看，主要是 Main 进程下的一个 Loop，然后按一定顺序执行各个子系统，并处理子系统的回传信号。其次是挂一个 monitor 进程。 2. 各个子系统通过一个**统一的消息总线和数据总线通信**，消息总线叫 DMS，数据总线叫 DS。 3. 回看 OpenWrt，也就是 NVMP 的编译框架，其特点是面向嵌入式设备设计，占用小，高度灵活和模块化。由此可见， NVMP **统一的消息总线和数据总线通信**的实现，主要是为了提高效率、降低占用，而不是使用层层包裹的通信方式。
周二	8.12	1. 培训：深度学习简介 2. 培训：轻量级神经网络 3. 测试样机 C260 连接 App 4. 因为还没有拿到串口板和开通串口，拜托同事给样机 C260 刷一个提测固件。	1. 基于大模型的数据生成方法，本质上像是用大模型训练小模型。能够明显扩充数据规模，从零生成新数据或是类似**数据增强**的方式得到新数据，主要用于比如 车辆识别，俯视角的目标检测，从而平衡数据集各个类别的规模。 2. 踩坑：尝试重置了很多次都重置失败，发现**拿到的测试样机的 Reset 键实际是损坏的**。Reset 按键有一定的反馈，但无法用于重置。 3. 第一次使用 Tapo 设备，使用蓝牙自动连接失败后，需要手动配置 Wi-Fi 连接。手动连接时，需要先将手机连接到设备 Wi-Fi TAPO_CAM_XXXX ，配置完网络密码后，再将手机连接至同一局域网 Wi-Fi。
周三	8.13	1. 阶段考核：代码基础 2. 反复测试 C260 未果，发现在自己没有串口线的情况下，测一台没有 reset 键的样机太难受了，更换为 Tapo C250. 3. 测试竞品样机和Tapo样机 4. 下音频实验室，使用标准测试流程测试 Tapo C250.	1. 先用30分钟在工位手动测了下 EZVIZ 和 Tapo 的对比视频，其实已经获取了大部分想要的信息了：两台样机的分辨率接近，录音效果接近，app 设计上 Tapo 明显领先；另外 EZVIZ 的 AI 功能一直在误报，顺藤摸瓜测了下几样声称的 AI 功能，发现漏报、误报率差不多到 50% 了（工位 1m 距离内）。 2. 第一次下实验室，很多地方都是第一次接触，包括第一次使用标准测试流程。基本保证了严谨性，但多个场景的测试结果导出后，花了很长时间分析。

	周四	8.14	1. 培训：前沿神经网络 2. 培训：高效模型结构设计及部署 3. 上午极限制作 ppt 4. 补全部分视频功能测试。	1. 因为原定当天下午答辩，我对自己的预期是 竞品分析 至少要把实验做了。因此取舍上 ppt 做的略微简陋，选取了一个双讲场景作为代表放了上去，完成了ppt初版，交导师。 2. 与导师讨论，结论是直播场景是基础场景，更适合作为代表场景。后来答辩时间宽裕了，决定按场景顺序，分别讨论 2 场景的实验结果。 3. 尽管在标准测试流程中还测了约 6 组数据，但讨论后决定只放两个场景的 4 组数据。
	周五	8.15	1. IPC产品分析撰写讲稿，口头排练。 2. 学习其他新人的 IPC产品分析 讲稿，发现了很多自己没考虑的点：包括实验结果附录、红框标注、更多的动图、视频实验结果。以及一些其他领域的知识：编码、呼吸效应、方块效应等等。	1. 对比实验室测试和实地测试，实验室的优点是控制变量效果更好，排除额外环境噪音后，数据结果一般也更好看；实地测试的优点是更符合用户场景，自由度高，实际的发现问题效率 有时更高。 2. 细化分工有两面性：一方面保证了我能在垂直领域有充足时间钻研，另一方面使人很容易丢失其他领域的视角。 Be curious, be humble.